K1Wi Framework

User Manual

Version: v0.99 RC1

Author:

Gregory B. Novak

Release Date:

June 2026

Document Information

Document Title:

K1Wi Framework User Manual

Version:

v0.99 RC1

Document Status:

Release Candidate

Target Audience:

Students

Researchers

Security Analysts

CTF Participants

Operating System:

Linux

About Document:

This manual provides operational guidance, usage examples, and technical reference material for the K1Wi Framework. It is intended for students, researchers, security analysts, and Capture-The-Flag participants seeking a unified toolkit for digital forensics, reverse engineering, binary analysis, and cryptographic investigations.

Table of Contents

Chapter 1 – What Is K1Wi?

1.1 Introduction

K1Wi is an integrated Cybersecurity analysis framework that combines reverse engineering, digital forensics, cryptanalysis, binary analysis, and Capture-The-Flag (CTF) tooling into a unified command-line environment.

The goal of K1Wi is to provide security professionals, students, researchers, and enthusiasts with a single toolkit capable of analyzing files, binaries, encoded data, cryptographic artifacts, and forensic evidence without requiring multiple disconnected utilities.

Unlike many specialized tools that focus on only one discipline, K1Wi brings together capabilities from several Cybersecurity domains. A user may extract files from an archive, inspect a Linux executable, analyze hidden data within an image, evaluate cryptographic material, calculate Position Independent Executable (PIE) offsets, and perform string intelligence analysis from the same framework.

1.2 Mission

The mission of K1Wi is to provide practical Cybersecurity analysis capabilities through a lightweight, transparent, and extensible platform.

K1Wi is designed to help users:

- Analyze suspicious files and artifacts
- Perform reverse engineering tasks
- Conduct digital forensic investigations
- Solve cryptographic and CTF challenges
- Inspect executable binaries
- Identify encoded, hidden, or suspicious data
- Learn Cybersecurity concepts through hands-on experimentation

1.3 Target Audience

K1Wi is intended for:

- Cybersecurity students
- Capture-The-Flag competitors
- Security researchers
- Reverse engineers
- Malware analysts
- Digital forensic investigators
- Incident responders

- Hobbyists and self-directed learners

No advanced programming knowledge is required to begin using K1Wi, although users with experience in operating systems, networking, programming, and information security will benefit from the framework's advanced capabilities.

1.4 Design Philosophy

K1Wi is built around several core principles:

1.4.1 Simplicity

Common tasks should require minimal commands and configuration.

1.4.2 Transparency

Analysis results should be understandable and traceable. K1Wi attempts to explain findings rather than simply report them.

1.4.3 Practicality

Features are selected based on real-world usefulness for security practitioners, students, and challenge solvers.

1.4.4 Modularity

Capabilities are organized into independent modules that can evolve without affecting the entire framework.

1.4.5 Education

K1Wi serves as both an operational toolkit and a learning platform for understanding Cybersecurity concepts.

1.5 Core Capability Areas

K1Wi currently provides functionality in the following areas:

1.5.1 String Intelligence

Detection and analysis of encoded, encrypted, structured, and suspicious strings.

1.5.2 Digital Forensics

Entropy analysis, file extraction, file carving, embedded signature detection, and artifact discovery.

1.5.3 Image Analysis

JPEG forensic analysis, entropy heat maps, RS steganography detection, Huffman fingerprinting, and DCT coefficient analysis.

1.5.4 Binary Analysis

ELF inspection, symbol enumeration, dependency analysis, and executable metadata extraction.

1.5.5 Reverse Engineering

PIE base calculations, symbol resolution, address translation, and runtime analysis support.

1.5.6 Cryptanalysis

RSA challenge analysis, factorization support, key inspection, and educational cryptographic tooling.

1.5.7 CTF Support

Integrated workflows that assist with challenge analysis, artifact inspection, and investigative problem solving.

1.6 Project Status

Current Release:

Version: v0.99 RC1

Current Development Status:

Release Candidate (RC)

K1Wi continues to evolve through additional analysis modules, expanded testing coverage, improved documentation, and user-driven enhancements.

The remainder of this manual describes installation, operation, workflows, command references, troubleshooting procedures, and development guidance for the K1Wi Framework.

Chapter 2 – Installation and Building K1Wi

2.1 Overview

This chapter describes the process of installing dependencies, compiling K1Wi from source, running tests, and verifying a successful installation.

K1Wi is currently developed and tested on Ubuntu Linux and other Debian-based distributions. The build system uses GNU Make and the Clang compiler.

2.2 System Requirements

Minimum recommended system:

- Ubuntu 22.04 LTS or newer
- 4 GB RAM
- 500 MB free disk space
- GCC or Clang compiler
- GNU Make

Recommended system:

- Ubuntu 24.04 LTS

- 8 GB RAM or greater
- Clang compiler
- Git
- AddressSanitizer support

2.3 Required Dependencies

K1Wi depends on several external libraries.

Install all required packages using:

```
sudo apt update

sudo apt install \
    build-essential \
    clang \
    make \
    libgmp-dev \
    libjpeg-dev \
    libssl-dev \
    libncurses-dev
```

2.4 Dependency Purpose

Package	Purpose
clang	Primary compiler
make	Build system
libgmp-dev	Large integer arithmetic used by RSA modules
libjpeg-dev	JPEG analysis and steganography modules
libssl-dev	Cryptographic functions
libncurses-dev	Terminal user interface support

2.5 Obtaining the Source Code

```
Clone the repository:
git clone https://github.com/GregNovak/K1Wi.git
cd K1Wi
```

2.6 Directory Structure

A typical K1Wi source tree contains:

```
software/
├── bin/
├── build/
├── Manuals/
```

```
├── tests/
├── testdata/
├── include/
├── *.c
├── *.h
├── makefile
└── README.md
```

2.6.1 Directory Descriptions

Directory	Purpose
bin	Compiled executables
build	Object files
Manuals	User documentation
tests	Regression test suite
testdata	Sample files used by testing
include	Shared header files

2.7 Building K1Wi

2.7.1 Clean Build

Remove previous build artifacts:

```
make clean
```

Compile the framework:

```
make
```

Expected output:

```
[CC] ...
[LINK] bin/k1wi
```

Upon successful completion, the executable will be located at:

```
bin/k1wi
```

2.8 Build Modes

2.8.1 Debug Build

Used during development.

```
make debug
```

Debug builds include symbols and additional diagnostic information.

2.8.2 Release Build

Used for production releases.

```
make release
```

Release builds prioritize optimization and reduced binary size.

2.8.3 Sanitizer Build

Used to detect memory safety issues.

```
make sanitize
```

Sanitizer builds help identify:

- Buffer overflows
- Use-after-free bugs
- Invalid memory access
- Memory corruption

2.8.4 Thread Sanitizer Build

Used to detect race conditions in multithreaded code.

```
make tsan
```

2.9 Verifying Installation

After compilation completes, verify that K1Wi starts correctly.

2.9.1 Display Version Information

```
./bin/k1wi --version
```

Expected output:

```
K1Wi Framework v0.99 RC1
Build Date: ...
Build Time: ...
```

2.9.2 Display Help Information

```
./bin/k1wi HELP
```

This command should display the available modules and command categories.

2.10 Running Regression Tests

K1Wi includes an automated regression suite.

Execute:

```
./tests/run_regression.sh
```

A successful run will end with output similar to:

```
============================================================
REGRESSION TEST SUMMARY
============================================================
PASS: 69
FAIL: 0
SKIP: 0
```

Any failures should be investigated before deploying or releasing K1Wi.

2.11 Running Sanitizer Validation

For maximum confidence before release:

```
make sanitize
./tests/run_regression.sh
```

A successful run confirms that the framework operates correctly under AddressSanitizer instrumentation.

2.12 Troubleshooting

2.12.1 Missing Header Files

Error:

```
fatal error: header.h: No such file or directory
```

Solution:

Verify that all required development packages are installed.

2.12.2 Missing Libraries

Error:

```
undefined reference to ...
```

Solution:

Verify installation of:

- libgmp-dev
- libjpeg-dev
- libssl-dev
- libncurses-dev

2.12.3 Permission Denied

Error:

```
Permission denied
```

Solution:

Verify executable permissions:

```
chmod +x ./bin/k1wi
```

2.12.4 Build Artifacts Corrupted

Solution:

Perform a clean rebuild:

```
make clean
make
```

2.13 Summary

At this point, K1Wi should compile successfully, pass regression testing, and be ready for operational use. The next chapter introduces the Quick Start Guide and demonstrates common workflows that can be performed within the first fifteen minutes of using K1Wi.

Chapter 3 – Quick Start Guide

3.1 Overview

This chapter introduces the most common K1Wi workflows. By the end of this chapter, users will be able to verify their installation, inspect files, analyze strings, inspect binaries, and execute the automated regression suite.

3.2 Verify Installation

Display version information:

```
./bin/k1wi --version
```

Expected output:

```
K1Wi Framework v0.99 RC1
Build Date: ...
Build Time: ...
Integrated Reverse Engineering and Cryptanalysis Framework
```

If version information appears, the executable is functioning correctly.

3.3 Display Available Commands

Show the built-in command reference:

```
./bin/k1wi HELP
```

This command displays the major K1Wi modules and supported workflows.

Important modules include:

- STRING

- EXTRACT
- LYZER
- ENTROPY
- ELFINFO
- PIECALC
- RSA

3.4 Analyze a Simple String

Command

```
./bin/k1wi STRING "hello world"
```

Example output:

```
Length: 11
Printable: yes
Entropy: ...
Detected Type: UTF-8 text
```

K1Wi automatically attempts to identify string types and detect common encodings.

3.5 Analyze a Suspicious String

Command

```
./bin/k1wi STRING "R2d2d2d2d2d424547494e20505249564154452044b45592d"
```

K1Wi will identify this as a shifted hexadecimal encoded private key marker.

This demonstrates the framework's ability to recognize encoded artifacts commonly encountered during investigations and CTF challenges.

3.6 Measure File Entropy

Command

```
./bin/k1wi ENTROPY <file>
```

Example output:

```
Global entropy: 7.638774 bits/byte
```

High entropy may indicate:

- Compression
- Encryption
- Packed executables
- Embedded payloads

3.7 Perform a Full Forensic Scan

Command

```
./bin/k1wi LYZER image.jpg ALL
```

This executes all available forensic modules, including:

- Entropy Heatmap
- RS Analysis
- Embedded Signature Detection
- File Carving
- String Intelligence
- JPEG Huffman Fingerprinting
- DCT Coefficient Analysis

This is one of the most powerful workflows within K1Wi.

3.8 Analyze an ELF Binary

Command

```
./bin/k1wi ELFINFO ./bin/k1wi
```

K1Wi will display:

- ELF header information
- Program headers
- Dynamic libraries
- Symbol tables

This functionality is useful for reverse engineering and binary analysis.

3.9 List PIE Symbols

Command

```
./bin/k1wi PIECALC --bin ./bin/k1wi --list
```

This displays symbol names and offsets useful for:

- ASLR analysis
- PIE calculations
- CTF challenges
- Exploit development research

3.10 Analyze an RSA Challenge

Command

```
./bin/k1wi RSA-FACTOR testdata/rsa/rsa_61_53_e17.txt
```

K1Wi will:

- Factor the modulus
- Recover the private exponent
- Decrypt the ciphertext
- Display recovered plaintext

This module is intended for educational cryptography and challenge-solving workflows.

3.11 Execute the Regression Suite

Command

```
./tests/run_regression.sh
```

Expected result:

```
PASS: 69
FAIL: 0
SKIP: 0
```

A successful run confirms that major framework functionality is operating correctly.

3.12 Summary

You have now completed the basic K1Wi workflow and verified the installation, command system, forensic analysis modules, binary analysis modules, cryptanalysis modules, and automated testing infrastructure.

The next chapter provides a detailed reference for the STRING analysis engine.

Chapter 4 – STRING Analysis Engine

4.1 Overview

The STRING module is K1Wi's universal string analysis engine. It is designed to identify, classify, and investigate textual data, encoded artifacts, and suspicious strings commonly encountered during cybersecurity investigations, digital forensics, reverse engineering, malware analysis, and Capture-The-Flag (CTF) challenges.

Unlike traditional string extraction utilities that simply display printable text, the STRING engine attempts to understand the content and identify patterns that may indicate encoded data, credentials, cryptographic material, URLs, email addresses, or other artifacts of interest.

4.2 Basic Usage

Analyze a string directly:

```
./bin/k1wi STRING "hello world"
```

Analyze a file:

```
./bin/k1wi STRING --file <file>
```

Decode detected content:

```
./bin/k1wi STRING --decode "<data>"
```

4.3 Supported Detection Types

The STRING engine currently supports detection of:

- UTF-8 text
- Printable ASCII text
- Hexadecimal strings
- Shifted hexadecimal strings
- Base64 content
- URLs
- Email addresses
- JWT artifacts
- Credential-like strings
- Encoded private key markers

Additional detectors may be added in future releases.

4.4 Example 1 – Plain Text

Input:

```
./bin/k1wi STRING "hello K1Wi"
```

Example output:

```
Length: 10
Printable: yes
Entropy: 3.1219
Detected Type: UTF-8 text
```

Interpretation:

The input appears to be normal human-readable text and does not exhibit characteristics commonly associated with encoded or encrypted content.

4.5 Example 2 – Shifted Hexadecimal Private Key Marker

Input:

```
./bin/k1wi STRING \
"R2d2d2d2d2d424547494e20505249564154454204b45592d"
```

Example output:

```
Detected Type: Shifted hex encoded private key
```

Interpretation:

The engine identified a shifted hexadecimal sequence that decodes to a recognizable private key header.

This capability can assist investigators in locating hidden cryptographic material embedded within files, memory dumps, archives, or challenge artifacts.

4.6 Entropy Analysis

Every analyzed string receives an entropy measurement.

Entropy is a statistical measure of randomness.

H=-\sum p_i\log_2 p_i

General guidelines:

Entropy	Interpretation
0-2	Highly repetitive
2-5	Typical text
5-7	Mixed or partially encoded
7-8	Compressed or encrypted

Entropy alone should not be used as proof of encryption or malicious activity.

4.7 String Intelligence

When analyzing files, K1Wi attempts to identify high-value artifacts.

Examples include:

- Potential credentials
- Encoded secrets
- Private key indicators
- URLs
- Email addresses
- Suspicious encoded content

Findings are categorized according to severity and confidence.

4.8 Limitations

The STRING module uses heuristic detection methods and may occasionally produce false positives or false negatives.

Users should validate significant findings through additional analysis.

4.9 Summary

The STRING engine serves as the primary entry point for text-based analysis within K1Wi and is often one of the first modules used during investigations, challenge solving, and forensic workflows.

Chapter 5 – EXTRACT Engine

5.1 Overview

The EXTRACT engine is K1Wi's recursive extraction and file-carving framework. It is designed to process archives, containers, embedded files, and nested artifacts commonly encountered during digital forensic investigations, malware analysis, incident response, and Capture-The-Flag (CTF) challenges.

Unlike traditional extraction tools that stop after processing a single archive, the EXTRACT engine can recursively process newly discovered artifacts and continue analysis through multiple layers of nesting.

The goal of the EXTRACT engine is to automate artifact discovery while maintaining visibility into the extraction process.

5.2 Core Capabilities

The EXTRACT engine supports:

- Recursive extraction
- Archive processing
- Embedded artifact discovery
- File carving
- String intelligence integration
- Session tracking
- Branch tracking
- Extraction reporting

5.3 Basic Usage

Extract a file:

```
./bin/k1wi EXTRACT sample.zip
```

Perform recursive extraction:

```
./bin/k1wi EXTRACT --recursive sample.zip
```

This instructs K1Wi to continue processing newly discovered files until no additional extractable content remains or the recursion limit is reached.

5.4 Extraction Sessions

Every extraction operation is assigned a unique session identifier.

Example:

```
EXTRACT SESSION 0001
```

The session number allows investigators to track results, correlate output directories, and maintain a record of extraction activity.

5.5 Extraction Workflow

The EXTRACT engine performs the following high-level workflow:

1. Validate input file
2. Identify file type
3. Extract supported content
4. Perform string intelligence analysis
5. Carve embedded artifacts
6. Queue newly discovered files
7. Repeat until completion

This workflow allows K1Wi to process deeply nested structures automatically.

5.6 Recursive Extraction

Recursive mode is one of the most powerful capabilities of the EXTRACT engine.

Example:

```
./bin/k1wi extract --recursive suspicious_archive.zip
```

Possible structure:

```
archive.zip
 ├── image.jpg
 ├── document.pdf
 └── hidden.zip
      ├── notes.txt
      └── payload.bin
```

When recursion is enabled, K1Wi continues processing newly discovered archives and artifacts until the extraction tree has been exhausted.

5.7 Output Directories

Example:

```
carved/extract_0001/
```

Within this directory, K1Wi stores:

- Extracted files
- Carved artifacts
- Intermediate processing results
- Final output artifacts

The directory structure preserves the extraction path and investigation history.

5.8 Branch Tracking

K1Wi tracks extraction branches to help investigators understand artifact lineage.

Example:

```
depth_00_branch_000_final
```

This naming convention allows users to determine:

- Recursion depth
- Branch origin
- Final artifact location

Branch tracking is especially useful when processing large or heavily nested archives.

5.9 String Intelligence Integration

During extraction, K1Wi automatically invokes STRING analysis on discovered artifacts.

Example output:

```
[STRING] file=sample.zip ASCII=5 UTF8=0 Base64=0 URL=0 Email=0
```

This provides investigators with immediate visibility into potentially useful content without requiring separate analysis steps.

5.10 File Carving

The EXTRACT engine can identify and recover embedded content discovered during analysis.

Potential examples include:

- ZIP archives
- JPEG images
- PDF documents
- PNG images
- GZIP streams

Recovered artifacts are automatically added to the extraction workflow.

5.11 Example Session

Command:

```
./bin/k1wi EXTRACT --recursive testdata/archives/sample.zip
```

Example output:

```
EXTRACT SESSION 0001

Target
------------------------------------------------------------
testdata/archives/sample.zip

Analysis
------------------------------------------------------------
[STRING] file=testdata/archives/sample.zip ASCII=5 UTF8=0

Output : carved/extract_0001/depth_00_branch_000_final
Files  : 2

EXTRACT COMPLETE
```

5.12 Practical Use Cases

The EXTRACT engine is useful for:

Malware Analysis

- Nested droppers
- Embedded payloads
- Packed archives

Digital Forensics

- Archive examination
- Evidence collection
- Artifact recovery

Incident Response

- Email attachment analysis
- Suspicious archive inspection
- Payload discovery

Capture-The-Flag Challenges

- Multi-stage archive challenges
- Hidden artifact discovery
- Embedded flag recovery

5.13 Limitations

Current limitations include:

- Recursion depth limits for safety
- Supported file types may vary by release
- Deeply nested archives may increase processing time
- False positives are possible during file carving

Users should validate significant findings independently.

5.14 Summary

The EXTRACT engine serves as the primary artifact discovery and recursive processing component of K1Wi. By combining extraction, carving, recursion, and string intelligence into a unified workflow, the engine provides investigators with an efficient method for locating hidden or nested content.

Chapter 6 – LYZER Forensic Analysis Engine

6.1 Overview

LYZER is K1Wi Framework's image analysis module designed to assist analysts in identifying embedded content, suspicious artifacts, steganography indicators, and hidden data within image files. LYZER combines multiple forensic techniques into a single workflow, including entropy analysis, string extraction, signature detection, file carving, steganography heuristics, JPEG structure analysis, and string intelligence.

The goal of LYZER is not to definitively prove that a file contains hidden information, but rather to identify indicators that warrant further investigation. By combining multiple analysis methods, LYZER provides analysts with a comprehensive view of the file and helps prioritize additional forensic examination.

6.2 Core Analysis Modules

The LYZER engine currently includes:

Table 6.1 – Core Analysis Modules

Module	Purpose
Entropy Analysis	Measures randomness
Entropy Heatmap	Identifies localized anomalies
RS Analysis	Detects steganographic indicators
Embedded Signature Detection	Locates hidden file types
File Carving	Recovers embedded artifacts
String Intelligence	Identifies useful textual artifacts
JPEG Huffman Fingerprinting	Detects custom Huffman tables
DCT Analysis	Evaluates JPEG coefficient distributions

6.3 Understanding LYZER Results

The purpose of LYZER is not to declare a file malicious. Instead, the engine highlights anomalies that may warrant additional investigation. Findings should be treated as indicators rather than proof. A single anomaly is rarely sufficient to reach a conclusion. Multiple independent findings generally increase confidence.

Table 6.2 – Entropy Interpretation

Entropy Range	Interpretation
0.0 - 2.0	Repetitive or mostly empty data
2.0 - 5.0	Typical text and structured content
5.0 - 7.0	Mixed content or moderate compression
7.0 - 8.0	Highly compressed or encrypted data
8.0	Maximum randomness

Table 6.3 – RS Analysis Interpretation

Observation	Possible Meaning
Balanced R/S values	Typical image
Significant R/S deviation	Potential steganography
Large flipped-group changes	Possible LSB embedding
Multiple anomalies	Increased suspicion
Normal RS profile	No strong indicator detected

6.4 Baseline Image Analysis

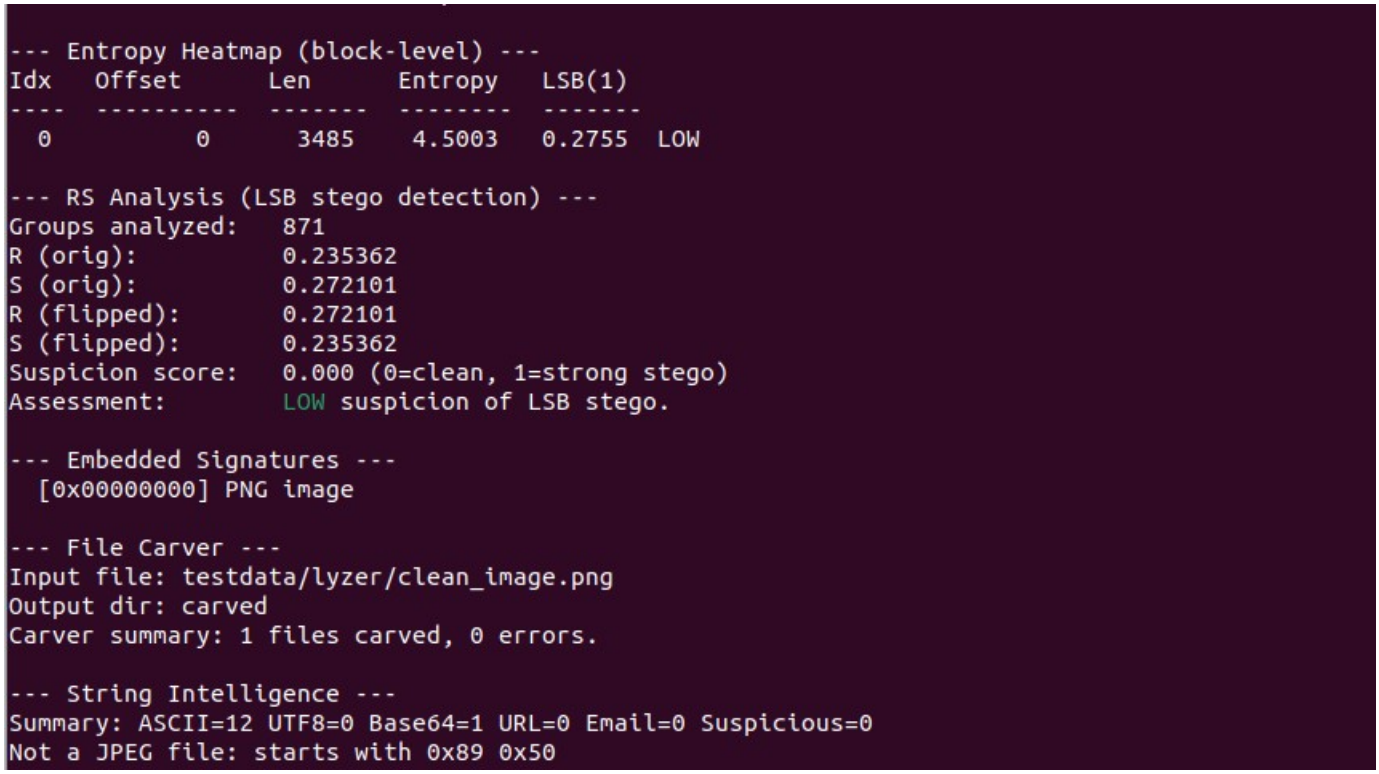
The first example uses a normal image file, *clean_image.png*, to establish a baseline for comparison. A clean image should contain no embedded archives, hidden payloads, or suspicious strings beyond what would normally be expected in an image file.

When analyzing a clean image, LYZER reports the file format, file size, entropy measurements, and any standard metadata discovered within the file. Entropy values are useful for identifying compressed or encrypted regions of data, while string extraction provides visibility into human-readable content contained within the file.

A baseline analysis allows the analyst to understand what normal output looks like before investigating more complex files that may contain hidden information.

6.4.1 Example 1 – Baseline Image Analysis

Figure 6.1 – Baseline Image Analysis



```
--- Entropy Heatmap (block-level) ---
Idx   Offset      Len      Entropy   LSB(1)
----  ----------  -------  --------  -------
  0          0      3485    4.5003   0.2755  LOW

--- RS Analysis (LSB stego detection) ---
Groups analyzed:   871
R (orig):          0.235362
S (orig):          0.272101
R (flipped):       0.272101
S (flipped):       0.235362
Suspicion score:   0.000 (0=clean, 1=strong stego)
Assessment:        LOW suspicion of LSB stego.

--- Embedded Signatures ---
  [0x00000000] PNG image

--- File Carver ---
Input file: testdata/lyzer/clean_image.png
Output dir: carved
Carver summary: 1 files carved, 0 errors.

--- String Intelligence ---
Summary: ASCII=12 UTF8=0 Base64=1 URL=0 Email=0 Suspicious=0
Not a JPEG file: starts with 0x89 0x50
```

Analyst Interpretation
The image was successfully identified as a PNG file. Entropy values were within expected ranges, no embedded files were detected, and String Intelligence reported no significant findings. This output establishes a baseline for comparison with more suspicious files.

6.5 Hidden Text and Secret Detection

The second example uses *k1wi_demo.jpg*, which contains intentionally appended text data for demonstration purposes. Although the image appears normal when opened in a standard image viewer, additional information exists beyond the end of the image data.

LYZER identifies indicators of suspicious content through its String Intelligence subsystem. During analysis, the module detected a high-value finding in the form of an embedded API key:

API_KEY=K1WI-DEMO-KEY-1234567890

This finding demonstrates how LYZER prioritizes potentially sensitive information rather than simply displaying large quantities of extracted text. The String Intelligence engine attempts to identify credentials, secrets, tokens, URLs, email addresses, and other high-value artifacts that may be useful during a forensic investigation.

The analysis also reported elevated steganography suspicion scores. Such findings do not prove the existence of hidden data but provide useful indicators that further examination may be warranted.

6.5.1 Example 2 – Hidden Text Detection

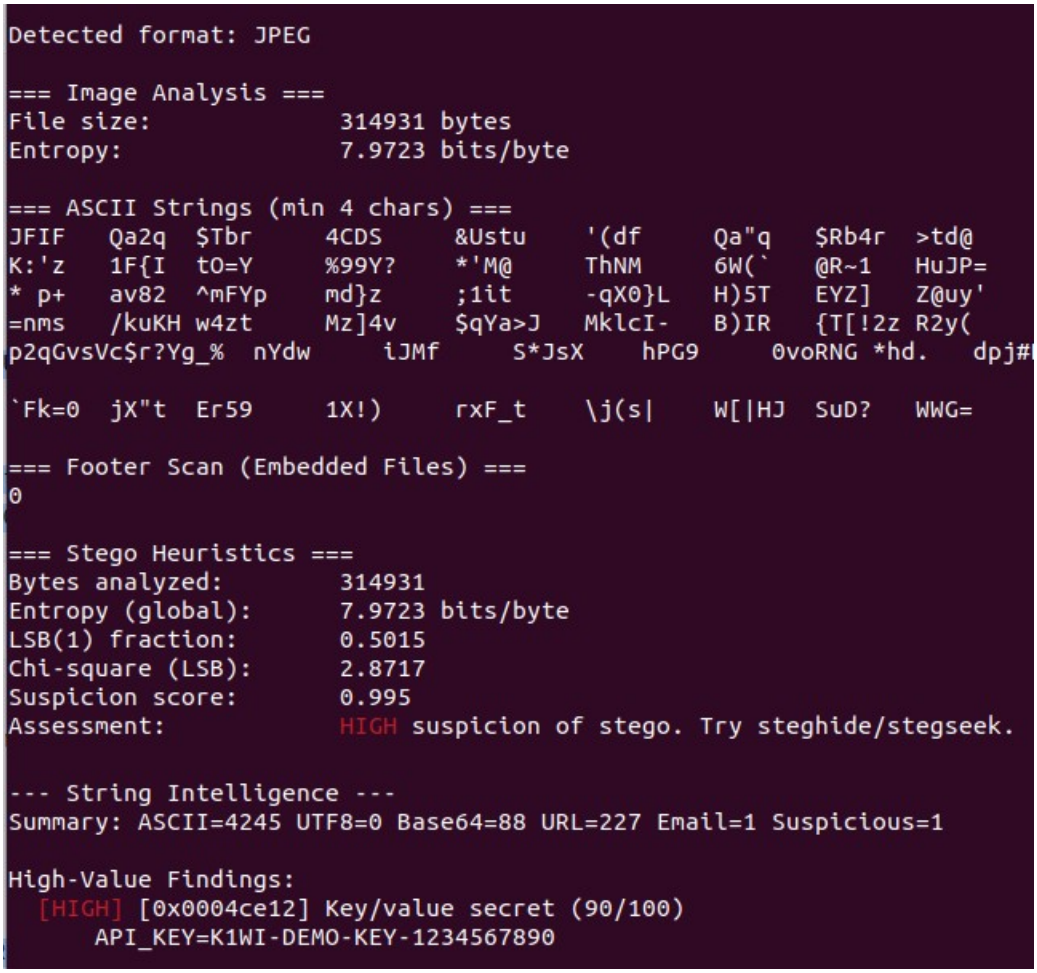


Figure 6.2 – Analysis of k1wi_demo.jpg. LYZER identified elevated steganography indicators and detected a high-value finding containing an embedded API key within the image.

Analyst Interpretation

Although the image appears normal when viewed with a standard image viewer, LYZER identified a hidden text payload containing an API key. The String Intelligence subsystem automatically prioritized the artifact as a high-value finding, allowing the investigator to quickly identify potentially sensitive information without manually reviewing thousands of extracted strings.

6.6 Embedded Archive Detection and File Carving

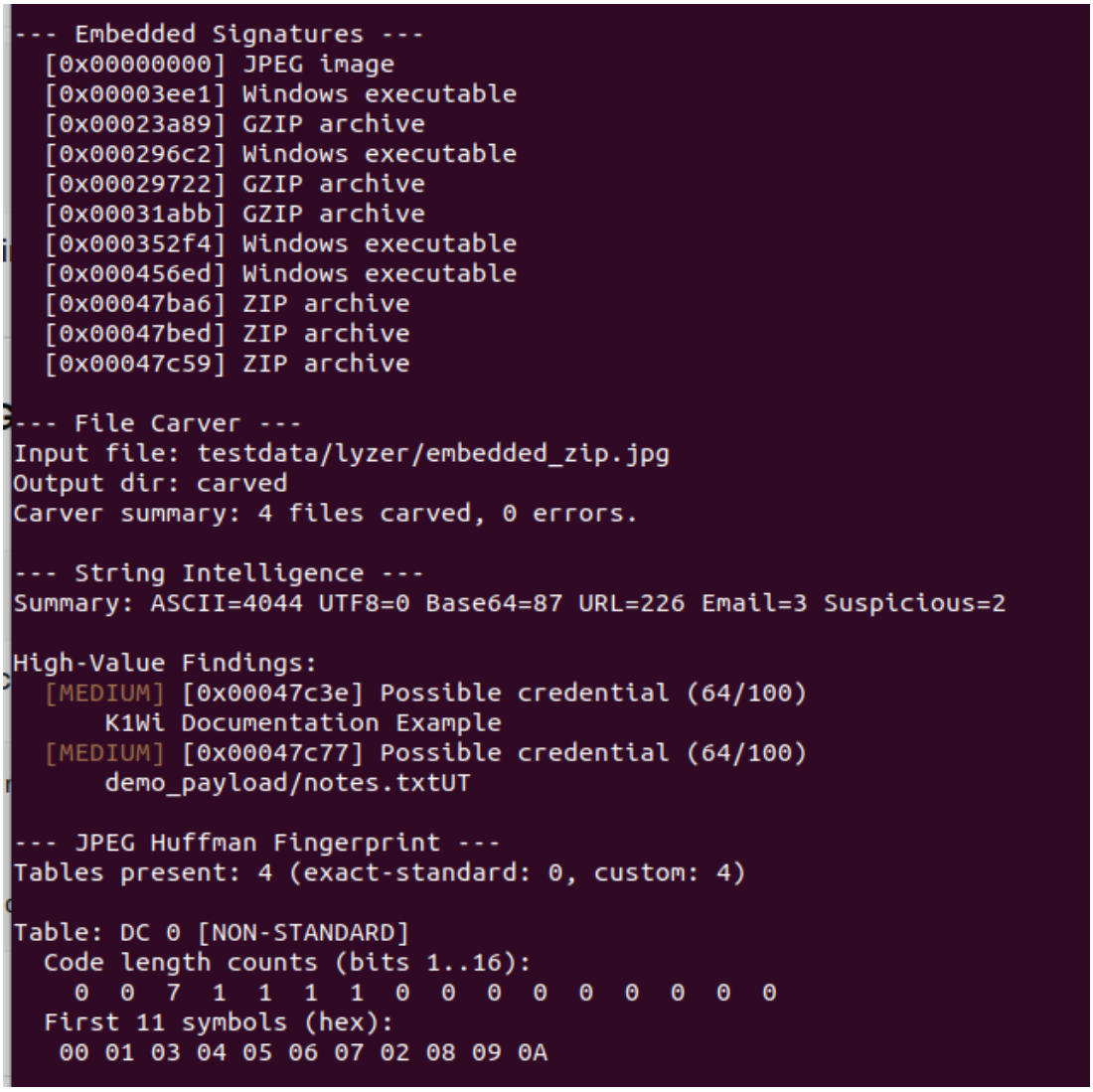
The final example uses *embedded_zip.jpg*, a JPEG image containing an appended ZIP archive. Although the image remains viewable as a standard JPEG, additional data exists beyond the end of the image file.

During analysis, LYZER detects embedded signatures associated with archive formats and invokes the file carving subsystem. The file carver extracts recoverable content and writes the recovered files to a dedicated output directory for additional examination.

This capability is particularly useful when investigating files that may contain appended archives, hidden payloads, malware droppers, or other embedded content designed to evade casual inspection.

Unlike simple string extraction, file carving attempts to recover complete embedded objects, allowing analysts to continue examining the recovered files with other K1Wi modules.

6.6.1 Example 3 – Embedded Archive Recovery



Analyst Interpretation

Although *embedded_zip.jpg* appears to be a normal JPEG image when viewed with standard image viewing software, LYZER identified an embedded ZIP archive contained within the file. The Embedded Signature Detection module located the archive, and the File Carver successfully recovered the embedded content for further examination.

This type of finding is significant because archives can be used to conceal documents, payloads, malware, or other artifacts that are not visible during normal file viewing. The presence of a recoverable archive increases the investigative priority of the file and warrants additional analysis of the recovered contents. In this example,

LYZER demonstrates its ability to identify hidden containers and extract embedded data that might otherwise go unnoticed during a routine inspection.

6.7 Advanced Analysis Modules

6.7.1 Embedded Signature Detection

The Embedded Signature Detection module searches files for known file headers and signatures. This capability helps identify hidden archives, documents, executables, and other embedded content that may not be visible through normal file viewing. When a valid signature is detected, investigators can use the File Carver module to recover the embedded object.

Table 6.4 – Embedded Signature Interpretation

Signature	Meaning
ZIP	Embedded archive
PDF	Embedded document
EXE	Embedded executable
JPEG/PNG	Embedded image
Unknown	Manual review required

6.7.2 JPEG Huffman Fingerprinting

JPEG Huffman Fingerprinting examines the Huffman tables used to compress JPEG images. Standard tables are commonly produced by digital cameras and image editing software, while custom tables may indicate recompression, specialized software, or image manipulation. This module provides an additional indicator that can assist investigators when evaluating suspicious images.

Table 6.5 – Huffman Fingerprint Results

Result	Interpretation
Standard	Normal JPEG
Mixed	Modified image
Custom	Recompression or custom encoder
Multiple Custom	Increased suspicion

6.7.3 DCT Coefficient Analysis

DCT Coefficient Analysis evaluates the distribution of JPEG compression coefficients. Unusual coefficient patterns may indicate image manipulation, recompression, or potential steganographic activity. Because legitimate image processing can also affect coefficient distributions, DCT analysis should be considered a supporting indicator rather than definitive proof of hidden content.

Table 6.6 – DCT Analysis Results

Result	Interpretation
Natural	Normal JPEG
Minor Deviations	Possible recompression
Significant Deviations	Further review recommended
Extreme Deviations	Possible manipulation

Table 6.7 – Confidence Levels

Confidence	Recommendation
LOW	Archive result
MEDIUM	Review manually
HIGH	Investigate further
CRITICAL	Immediate analyst review

Table 6.8 – Investigator Actions

Finding	Action
High Entropy	Review heatmap
RS Anomaly	Test for stego
Signature Found	Run carver
Secret Found	Investigate
Archive Found	Analyze recovered files

6.8 Interpreting Results

No single indicator should be considered proof of malicious activity. Analysts should evaluate entropy measurements, string intelligence findings, embedded signatures, file carving results, and steganography heuristics collectively.

For example, a file may exhibit high entropy because it contains compressed image data rather than hidden information. Likewise, a steganography heuristic may generate a false positive on a legitimate image. LYZER is designed to provide investigative leads and indicators rather than definitive conclusions.

The most effective approach is to combine multiple findings and follow up with additional analysis when suspicious artifacts are identified.

6.9 Summary

LYZER provides a unified workflow for image-based forensic analysis. Through entropy measurements, string intelligence, embedded signature detection, file carving, steganography analysis, and JPEG structure inspection, the module helps analysts identify potentially hidden information that may not be visible through conventional image viewing tools.

The examples presented in this chapter demonstrate three common scenarios:

1. Baseline analysis of a normal image.

2. Detection of hidden text and sensitive information.
3. Recovery of embedded archives through file carving.

Together, these examples illustrate how LYZER can assist analysts in identifying suspicious content and prioritizing further forensic investigation.

Chapter 7 – PIECALC Engine

7.1 Overview

PIECALC is a reverse engineering utility designed to assist analysts working with Position Independent Executables (PIE). Modern Linux binaries frequently use Address Space Layout Randomization (ASLR), causing function addresses to change each time a program executes. PIECALC simplifies this process by calculating runtime addresses, locating symbols, and identifying the nearest known function to an arbitrary address.

The module is intended for reverse engineering, exploit development, debugging, and binary analysis workflows. By combining symbol information with runtime address calculations, PIECALC helps analysts quickly translate leaked addresses into meaningful program locations.

7.2 Understanding PIE Binaries

Position Independent Executables do not execute at fixed memory addresses. Instead, the operating system loads the binary at a randomized base address during program startup. As a result, addresses observed during debugging, crash analysis, or exploitation must often be translated back to their original symbol offsets.

PIECALC automates these calculations and provides a convenient interface for symbol lookup and address resolution.

7.3 Core Features

Table 7.1 – PIECALC Core Features

Feature	Description
Symbol Listing	Displays available function symbols and offsets
Base Address Calculation	Computes runtime addresses from offsets
Nearest Symbol Lookup	Identifies the closest known symbol to an address
Offset Resolution	Converts runtime addresses to symbol offsets
JSON Output	Generates machine-readable output
ELF Integration	Works directly with ELF symbol information

7.4 Basic Usage

PIECALC can be used directly from the K1Wi command line.

Example:

```
PIECALC --bin ./bin/k1wi --list
```

This command displays the available symbols discovered within the target binary.

Example:

```
PIECALC --bin ./bin/k1wi --nearest 0xdce0
```

This command locates the nearest known symbol to the supplied address.

Example:

```
PIECALC --bin ./bin/k1wi --json --list
```

This command produces machine-readable output suitable for scripting and automation.

7.5 Example 1 – Listing Symbols

Command

```
PIECALC --bin ./bin/k1wi --list
```

Purpose

The `--list` option displays the symbols discovered within the target binary. This information provides analysts with a quick overview of available functions and their offsets. Symbol listings are useful when examining program structure, identifying candidate functions, and performing reverse engineering tasks.

Figure 7.1 – PIECALC Symbol Enumeration

PIECALC listing function symbols and offsets recovered from the k1wi executable.

Analyst

```
Command: PIECALC --bin ./bin/k1wi --list
fill_huff_summary         0xa090
matches_standard_table    0xa1e0
init_app                  0xa810
init_ui                   0xa970
set_panel                 0xa9c0
draw_banner               0xaae0
draw_menu_bar             0xac00
draw_panel                0xadd0
draw_status_bar           0xb030
handle_global_key         0xb160
shutdown_ui               0xb3f0
cleanup_app               0xb400
load_directory_files      0xba40
init_colors               0xbd90
draw_centered             0xbe40
draw_panel_menu           0xbf30
draw_panel_file           0xbfb0
draw_panel_vigan          0xc260
```

Interpretation

PIECALC can enumerate function symbols contained within an ELF executable and display their corresponding offsets. This capability allows analysts to quickly identify application routines, cryptographic functions, forensic modules, and internal PIECALC helper functions. Symbol enumeration is useful during reverse engineering, exploit development, binary triage, and debugging because it provides a map of executable functionality without requiring manual inspection of the binary. In this example, PIECALC successfully identifies both application-level functions and internal analysis routines from the K1Wi executable.

7.6 Example 2 – Nearest Symbol Lookup

Command

```
PIECALC --bin ./bin/k1wi --nearest 0xdce0
```

Purpose

The --nearest option identifies the closest known symbol to a supplied address. This capability is particularly useful when analyzing crash reports, memory leaks, debugger output, or leaked addresses obtained during reverse engineering.

Figure 7.2 – PIECALC Nearest Symbol Lookup

PIECALC identifying the nearest known symbol to address 0xdce0.

```
Command: PIECALC --bin ./bin/k1wi --nearest 0xdce0
[NEAREST SYMBOLS]
  solve_monoalphabetic     off=0xdc00 dist=0xe0
  run_vigan_analysis       off=0xdae0 dist=0x200
  init_random_key          off=0xe080 dist=0x3a0
  load_file_preview        off=0xd8e0 dist=0x400
  copy_state               off=0xe130 dist=0x450
```

Analyst Interpretation

Addresses observed during execution often do not correspond exactly to the beginning of a function. Nearest symbol lookup provides context by identifying the closest known function and its offset relationship. This allows analysts to quickly determine which portion of the program generated the observed address.

In this example, PIECALC correctly identifies guess_leak_symbol as the nearest function to address 0xdce0 with a distance of zero bytes, indicating an exact match. Additional nearby functions are also shown, allowing analysts to quickly understand the surrounding program structure.

This capability is particularly useful during crash analysis, exploit development, debugger investigations, and reverse engineering workflows where only partial address information may be available.

7.7 Example 3 – JSON Output

Command

```
PIECALC --bin ./bin/k1wi --json --list
```

Purpose

The JSON output mode generates machine-readable results suitable for scripting, automation, and integration with other analysis tools. Structured output enables analysts to process PIECALC results programmatically without relying on manual parsing.

Figure 7.3 – PIECALC JSON Output

PIECALC exporting symbol information in JSON format for automation, scripting, and integration with external analysis tools.

```
Command: PIECALC --bin ./bin/k1wi --json --list
{
  "binary": "./bin/k1wi",
  "symbols": [
    {"name": "fill_huff_summary", "offset": "0xa090"},
    {"name": "matches_standard_table", "offset": "0xa1e0"},
    {"name": "init_app", "offset": "0xa810"},
    {"name": "init_ui", "offset": "0xa970"},
    {"name": "set_panel", "offset": "0xa9c0"},
    {"name": "draw_banner", "offset": "0xaae0"},
    {"name": "draw_menu_bar", "offset": "0xac00"},
    {"name": "draw_panel", "offset": "0xadd0"},
    {"name": "draw_status_bar", "offset": "0xb030"},
    {"name": "handle_global_key", "offset": "0xb160"},
    {"name": "shutdown_ui", "offset": "0xb3f0"},
```

Analyst Interpretation

JSON output is particularly useful when PIECALC is integrated into automated workflows. Security researchers, malware analysts, and reverse engineers can incorporate PIECALC results into custom scripts, dashboards, or analysis pipelines while maintaining consistent output formatting.

Table 7.2 – PIECALC Output Fields

Field	Description
Symbol Name	Name of the discovered function
Offset	Relative offset within the binary
Runtime Address	Calculated runtime address
Distance	Difference between target address and nearest symbol
Binary	Target executable being analyzed
Matches	Symbols returned by the query

7.8 Practical Reverse Engineering Workflow

A typical reverse engineering workflow begins by identifying available symbols within a target executable. Analysts can use PIECALC to enumerate symbols, locate addresses of interest, and correlate runtime observations with known program functions. When combined with debuggers, disassemblers, and ELF analysis tools, PIECALC provides a rapid method for translating raw addresses into actionable intelligence.

PIECALC is often used alongside ELFINFO, GDB, objdump, and other binary analysis tools to rapidly translate leaked addresses into meaningful program context.

Common use cases include:

1. Enumerating function symbols.
2. Identifying the nearest function to a leaked address.
3. Correlating debugger output with program symbols.
4. Generating structured output for automated analysis.

5. Supporting exploit development and binary research activities.

7.9 Summary

In this chapter, the PIECALC Engine was used to:

1. Enumerate symbols within PIE-enabled binaries.
2. Locate the nearest symbol to an arbitrary address.
3. Generate machine-readable JSON output.
4. Assist reverse engineering and debugging workflows.
5. Translate raw address information into meaningful program context.

The PIECALC Engine provides a practical interface for working with modern position-independent executables and serves as a valuable tool for binary analysis and reverse engineering tasks.

Chapter 8 – ELFINFO Engine

8.1 Overview

ELFINFO is K1Wi's ELF binary inspection utility. The command allows analysts to examine executable files, shared libraries, and other ELF-based binaries without requiring external tools such as readelf or objdump.

ELFINFO extracts information directly from ELF headers and structures to provide insight into binary architecture, program layout, imported libraries, and symbol information. This capability is useful during malware analysis, reverse engineering, vulnerability research, exploit development, and general binary inspection.

The command supports both 32-bit and 64-bit ELF files and presents information in a concise analyst-friendly format.

8.2 Understanding ELF Files

The Executable and Linkable Format (ELF) is the standard binary format used by Linux and many Unix-like operating systems. ELF files contain executable code, metadata, memory layout information, imported libraries, debugging information, and symbol tables.

When a program is executed, the operating system uses information stored within the ELF file to determine how the binary should be loaded into memory.

Key ELF structures include:

- ELF Header
- Program Headers
- Section Headers
- Dynamic Linking Information
- Symbol Tables

ELFINFO provides visibility into each of these structures to help analysts understand how a binary operates.

8.3 Core Features

ELFINFO provides the following capabilities:

- ELF header analysis
- Program header enumeration
- Dynamic library identification
- Symbol table extraction
- 32-bit and 64-bit ELF support
- Reverse engineering support
- Dependency analysis
- Binary reconnaissance

These features allow analysts to quickly identify binary characteristics without requiring multiple external utilities.

8.4 Basic Usage

Command

```
ELFINFO -IN <binary>
```

Example

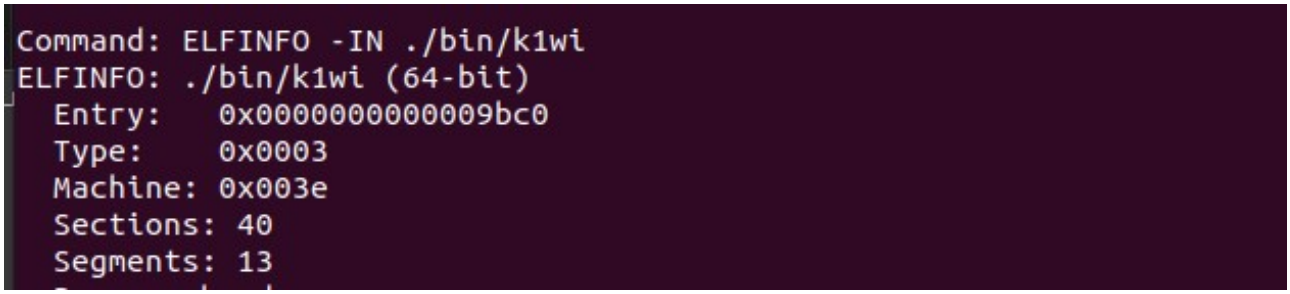
```
./bin/k1wi ELFINFO -IN ./bin/k1wi
```

Purpose

The ELFINFO command reads and displays structural information from the specified ELF binary. This information can be used to identify architecture details, imported libraries, memory layout information, and available symbols.

Figure 8.1 – Basic ELFINFO Analysis

ELFINFO -IN ./bin/k1wi
ELF Header Output



ELFINFO displaying architecture, entry point, file type, and structural information for the K1Wi executable.

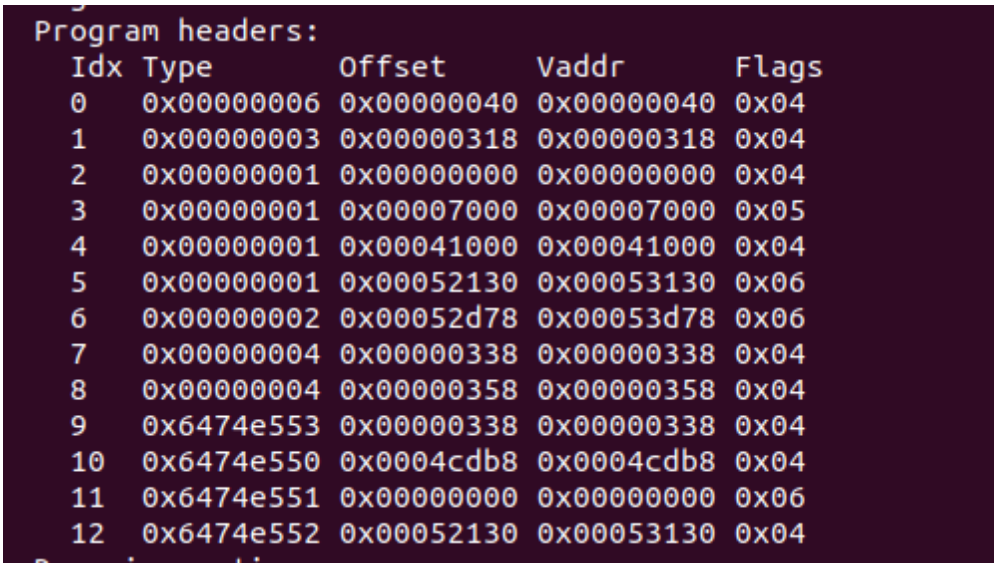
Analyst Interpretation

The ELF header provides fundamental information about a binary file. This information includes the executable type, target architecture, entry point address, and the number of sections and program segments contained within the file. Analysts frequently begin ELF investigations by reviewing the ELF header because it provides a high-level overview of how the binary is organized.

In this example, ELFINFO identifies the K1Wi executable as a 64-bit ELF binary and displays its entry point and structural characteristics. This information helps analysts quickly determine the binary's architecture and execution environment before performing deeper analysis.

Figure 8.2 – Program Header Analysis

ELFINFO displaying program header entries used by the operating system loader during program execution.



```
Program headers:
  Idx Type       Offset     Vaddr      Flags
  0   0x00000006 0x00000040 0x00000040 0x04
  1   0x00000003 0x00000318 0x00000318 0x04
  2   0x00000001 0x00000000 0x00000000 0x04
  3   0x00000001 0x00007000 0x00007000 0x05
  4   0x00000001 0x00041000 0x00041000 0x04
  5   0x00000001 0x00052130 0x00053130 0x06
  6   0x00000002 0x00052d78 0x00053d78 0x06
  7   0x00000004 0x00000338 0x00000338 0x04
  8   0x00000004 0x00000358 0x00000358 0x04
  9   0x6474e553 0x00000338 0x00000338 0x04
  10  0x6474e550 0x0004cdb8 0x0004cdb8 0x04
  11  0x6474e551 0x00000000 0x00000000 0x06
  12  0x6474e552 0x00052130 0x00053130 0x04
```

Analyst Interpretation

Program headers describe how an executable file should be mapped into memory when it is loaded by the operating system. Each entry contains information about memory addresses, file offsets, permissions, and segment types.

Program header analysis is useful during reverse engineering and exploit development because it reveals executable regions, writable areas, and memory layout characteristics. Understanding how a binary is loaded into memory can assist analysts when investigating crashes, memory corruption vulnerabilities, or runtime behavior.

8.5 Example 1 – Dynamic Library Enumeration

Command

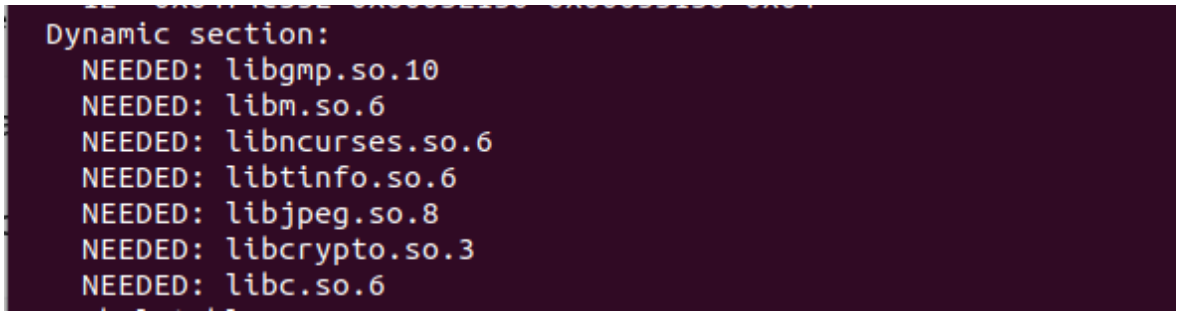
ELFINFO -IN ./bin/k1wi

Purpose

Dynamic library enumeration identifies external libraries required by the executable at runtime. This information helps understand software dependencies, imported functionality, and potential attack surface.

Figure 8.3 – Dynamic Library Enumeration

ELFINFO displaying runtime library dependencies required by the K1Wi executable.



Analyst Interpretation

Dynamic libraries reveal the external dependencies required by a binary at runtime. Analysts frequently examine imported libraries to understand application functionality, identify attack surface, detect suspicious dependencies, and evaluate software composition.

Library enumeration is particularly useful during malware analysis, reverse engineering, software auditing, and vulnerability research because imported libraries often reveal major application capabilities. Networking libraries may indicate communication functionality, cryptographic libraries may indicate encryption support, and compression libraries may indicate archive handling capabilities.

In this example, ELFINFO identifies the runtime libraries required by the K1Wi executable and provides visibility into the software components needed during execution.

8.6 Example 2 – Symbol Table Analysis

Command

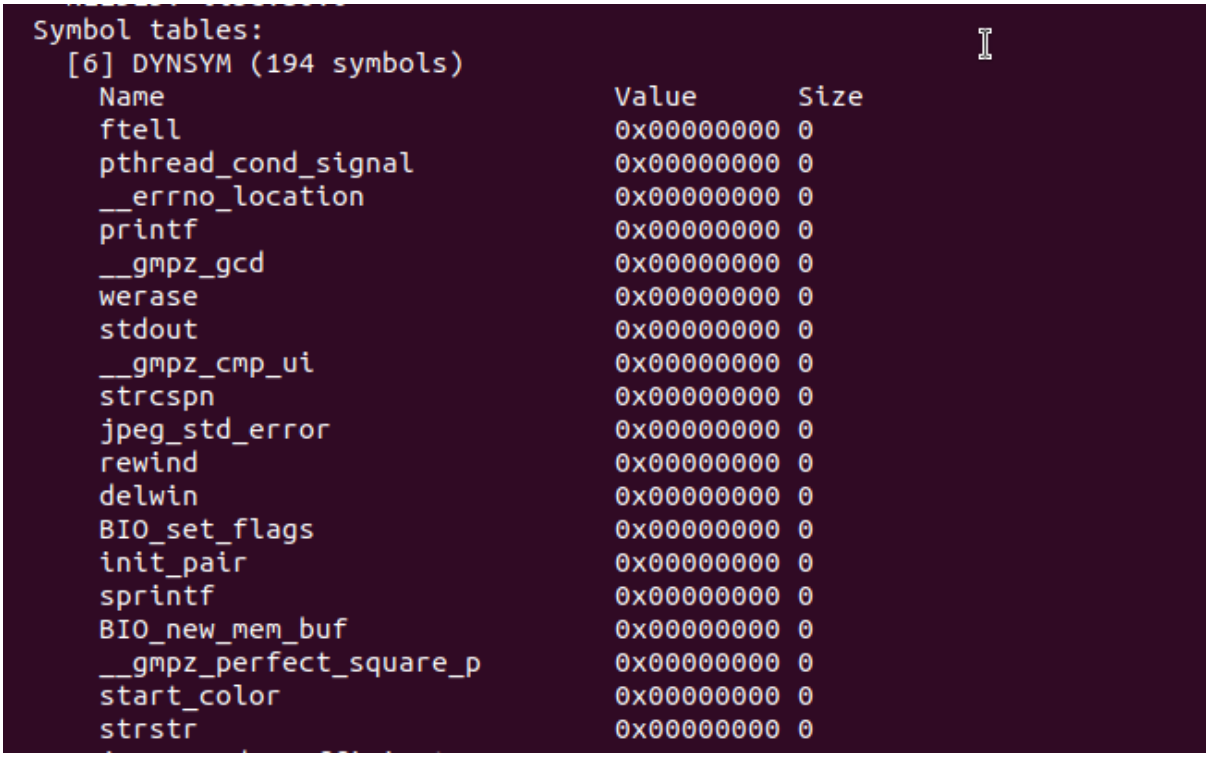
./bin/k1wi ELFINFO -IN ./bin/k1wi

Purpose

Symbol table analysis displays imported and exported symbols referenced by the executable. This information helps analysts identify functions, variables, library calls, and program behavior.

Figure 8.4 – Symbol Table Analysis

ELFINFO displaying imported and exported symbols contained within the executable.



```
Symbol tables:
  [6] DYNSYM (194 symbols)
    Name                    Value      Size
    ftell                   0x00000000 0
    pthread_cond_signal     0x00000000 0
    __errno_location        0x00000000 0
    printf                  0x00000000 0
    __gmpz_gcd              0x00000000 0
    werase                  0x00000000 0
    stdout                  0x00000000 0
    __gmpz_cmp_ui           0x00000000 0
    strcspn                 0x00000000 0
    jpeg_std_error          0x00000000 0
    rewind                  0x00000000 0
    delwin                  0x00000000 0
    BIO_set_flags           0x00000000 0
    init_pair               0x00000000 0
    sprintf                 0x00000000 0
    BIO_new_mem_buf         0x00000000 0
    __gmpz_perfect_square_p 0x00000000 0
    start_color             0x00000000 0
    strstr                  0x00000000 0
```

Analyst Interpretation

Symbol tables provide visibility into functions, variables, and imported routines referenced by an executable. This information assists reverse engineers, malware analysts, and software auditors when examining binary functionality.

8.7 Practical ELF Analysis Workflow

A typical ELF investigation begins by examining the ELF header to identify architecture, executable type, and entry point information. Analysts then review program headers to understand memory layout, permissions, and runtime loading behavior. Dynamic library enumeration provides insight into external dependencies and potential attack surface.

Finally, symbol table analysis reveals imported and exported functionality that can assist reverse engineering and software auditing efforts.

Common use cases include:

- Malware triage
- Dependency analysis
- Reverse engineering
- Binary reconnaissance
- Vulnerability research

8.8 Summary

Throughout this chapter, ELFINFO was used to examine ELF headers, program headers, dynamic libraries, and symbol tables contained within Linux executables.

The examples demonstrated how ELFINFO can assist with binary reconnaissance, dependency analysis, reverse engineering, malware investigation, and software auditing.

By providing visibility into executable structure, imported libraries, and runtime dependencies, ELFINFO serves as a practical first-step analysis tool for understanding unfamiliar ELF binaries.

The next chapter introduces the RSA Analysis Tool Suite and demonstrates a variety of cryptographic analysis and key recovery workflows.

Chapter 9 – RSA Analysis Tools

9.1 Overview

The RSA Analysis Tool Suite provides a collection of educational and investigative modules for analyzing RSA cryptographic challenges. These tools are designed to assist students, security researchers, Capture-The-Flag (CTF) participants, and reverse engineers when examining RSA implementations, recovering keys, validating parameters, and solving cryptographic exercises.

Unlike general-purpose cryptographic libraries, the K1Wi RSA modules focus on transparency and education. Each tool exposes intermediate calculations and recovery steps to help users understand how RSA attacks and key recovery techniques operate.

The RSA tool suite currently includes:

- RSA-FACTOR – Classical and Fermat factorization
- RSA-RHO – Pollard Rho factorization
- RSA-ECM – Elliptic Curve Method factorization
- RSA-WIENER – Wiener's continued fraction attack

- RSA-SMALL-E – Low public exponent attack
- RSA-KNOWNPQ – Decrypt using known prime factors
- RSA-CHECKPQ – Validate candidate RSA primes
- RSA-DFROMPQ – Recover the private exponent
- RSA-MINI – Educational RSA challenge helper

These tools are intended for educational use, challenge solving, cryptographic research, and controlled security testing environments.

9.2 Understanding RSA

RSA is a public-key cryptographic system based on the mathematical difficulty of factoring large composite integers.

An RSA key pair consists of:

- Prime number p
- Prime number q
- Modulus $n = p \times q$
- Public exponent e
- Private exponent d

The public key contains n and e.

The private key contains d and is derived using Euler's Totient Function:

$$\varphi(n) = (p - 1)(q - 1)$$

The private exponent is calculated as:

$$d \equiv e^{-1} \bmod \varphi(n)$$

Encryption:

$$c = m^e \bmod n$$

Decryption:

$$m = c^d \bmod n$$

Many RSA attacks rely on weaknesses in key generation, unusually small parameters, known factors, or implementation mistakes. The K1Wi RSA modules demonstrate several of these techniques.

9.3 RSA-FACTOR

Purpose

RSA-FACTOR performs classical RSA modulus factorization using Fermat and related factorization techniques. When successful, the tool recovers the prime factors of the modulus, computes the private exponent, and attempts plaintext recovery.

This module is useful for educational RSA challenges, weak-key demonstrations, and Capture-The-Flag (CTF) exercises where the RSA modulus is intentionally vulnerable.

Usage

```
./bin/k1wi RSA-FACTOR <rsa_file>
```

Example

```
./bin/k1wi RSA-FACTOR testdata/rsa/rsa_61_53_e17.txt
```

Figure 9.1 – RSA-FACTOR Recovery

Analyst Interpretation

The RSA modulus is:

N = 3233

The factorization process successfully recovers:

p = 53
q = 61

Using these values, K1Wi computes Euler's Totient Function:

$\varphi(N) = (p - 1)(q - 1)$
$\varphi(N) = 3120$

The private exponent is then derived:

d = 2753

Once the private key components are recovered, the ciphertext is decrypted and the plaintext value is displayed.

This example demonstrates the complete RSA recovery workflow, including modulus factorization, private key reconstruction, and plaintext recovery from a vulnerable RSA challenge file.

Investigation Notes

RSA-FACTOR is most effective against:

- Small RSA challenge keys
- Educational RSA examples
- Weakly generated RSA moduli
- CTF cryptography exercises

Modern RSA implementations use significantly larger key sizes and properly generated primes. Such keys are not expected to be vulnerable to simple Fermat factorization techniques.

Practical Applications

RSA-FACTOR can be used to:

- Recover private keys from weak RSA challenges
- Demonstrate RSA key recovery concepts
- Validate RSA training exercises
- Support cryptography coursework
- Solve introductory CTF RSA problems

Figure 9.1 demonstrates successful factorization, private key recovery, and plaintext extraction from a vulnerable RSA challenge file.

9.4 RSA-RHO

Purpose

RSA-RHO implements Pollard Rho factorization, a probabilistic integer factorization algorithm commonly used against small or weak RSA moduli.

Unlike Fermat factorization, which performs best when the prime factors are close together, Pollard Rho uses pseudo-random sequences to search for non-trivial factors. This makes it effective against certain RSA challenges where classical factorization methods may be less efficient.

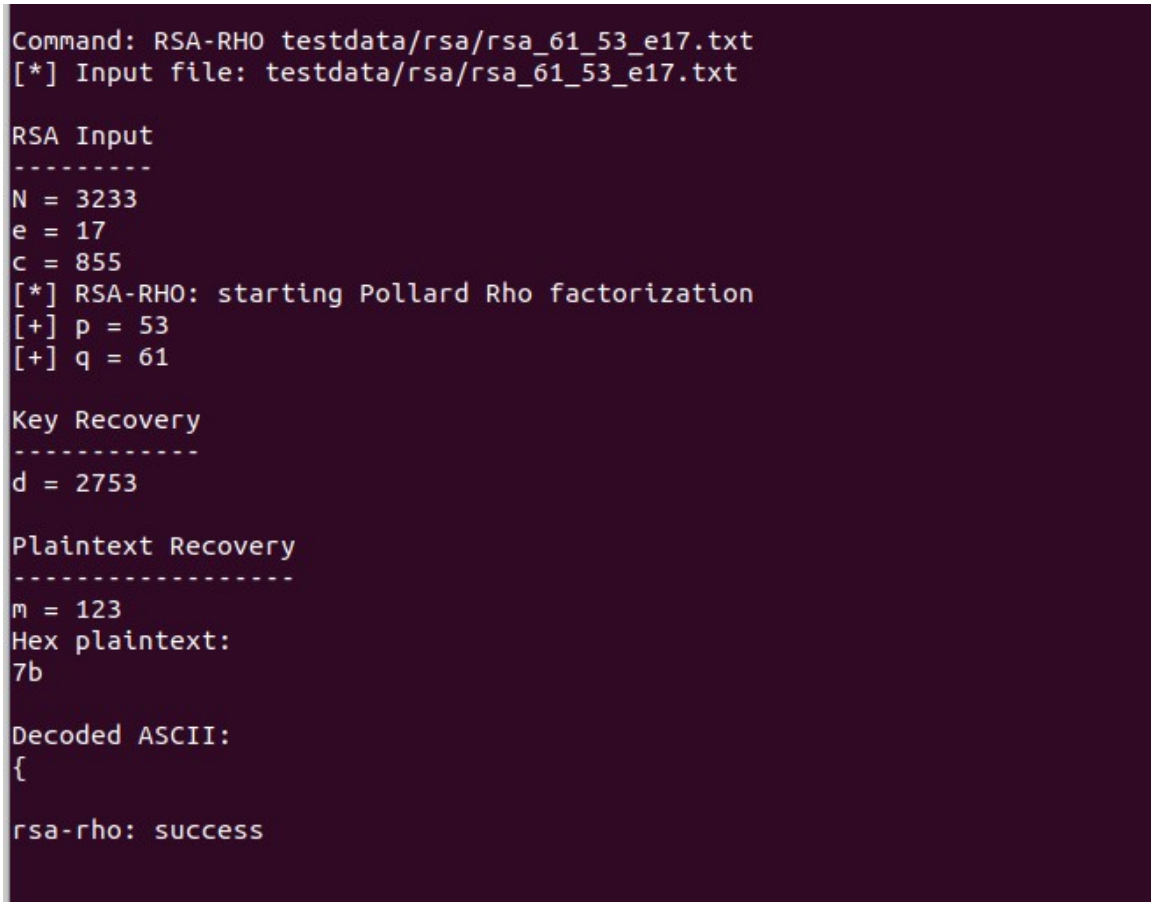
Usage

```
./bin/k1wi RSA-RHO <rsa_file>
```

Example

```
./bin/k1wi RSA-RHO testdata/rsa/rsa_61_53_e17.txt
```

Figure 9.2 – RSA-RHO Factorization Workflow



RSA-RHO using Pollard Rho factorization techniques to recover non-trivial factors from a vulnerable RSA challenge file.

Analyst Interpretation

Pollard Rho attempts to discover a non-trivial factor of the RSA modulus using iterative modular arithmetic and greatest common divisor (GCD) calculations.

Unlike Fermat factorization, which is most effective when the prime factors are relatively close together, Pollard Rho uses a probabilistic search strategy that can occasionally recover factors more efficiently.

Once a factor is recovered, K1Wi can derive the remaining RSA parameters and continue the key recovery process. This makes RSA-RHO a useful secondary attack when RSA-FACTOR is unsuccessful or when alternative factorization methods are being evaluated.

The algorithm derives its name from the characteristic "ρ" (rho) shape often observed when visualizing sequence cycles.

Investigation Notes

RSA-RHO is most effective against:

- Small RSA challenge keys
- Educational RSA exercises
- CTF cryptography challenges
- Weakly generated RSA moduli

Pollard Rho is generally faster than brute-force factorization and is frequently used as an initial attack when evaluating weak RSA keys.

Practical Applications

RSA-RHO can be used to:

- Recover factors from weak RSA moduli
- Demonstrate probabilistic factorization techniques
- Compare factorization methods against the same challenge file
- Support cryptography training and coursework

9.5 RSA-ECM

Purpose

RSA-ECM implements the Elliptic Curve Method (ECM) of integer factorization. ECM is a powerful factorization technique developed by Hendrik Lenstra and is particularly effective when one of the RSA prime factors is significantly smaller than the other.

Compared to classical factorization and Pollard Rho, ECM can successfully recover factors from some RSA challenge moduli that resist simpler approaches.

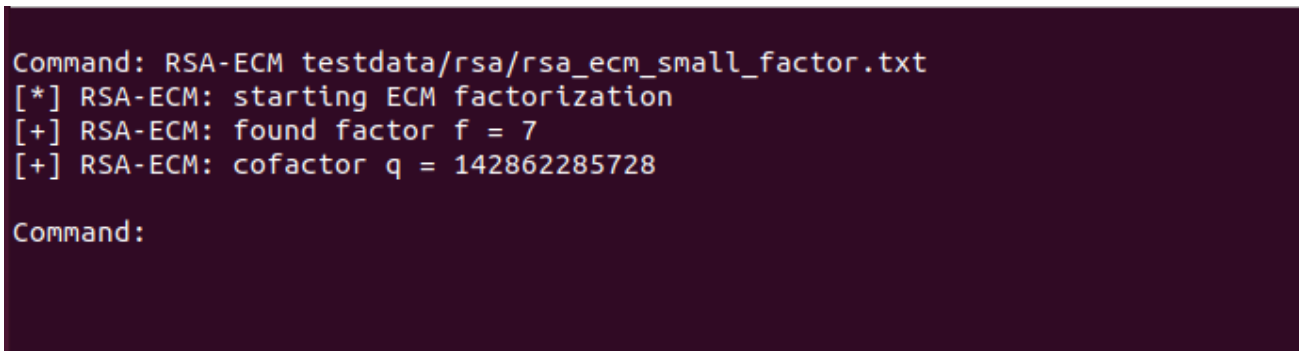
Usage

./bin/k1wi RSA-ECM <rsa_file>

Example

./bin/k1wi RSA-ECM testdata/rsa/rsa_ecm_small_factor.txt

Figure 9.3 – RSA-ECM Factorization



RSA-ECM using elliptic curve factorization techniques to recover a non-trivial factor from a vulnerable RSA challenge file.

Analyst Interpretation

The Elliptic Curve Method uses arithmetic performed on randomly selected elliptic curves over finite fields to discover non-trivial factors of an RSA modulus.

Unlike RSA-FACTOR and RSA-RHO, ECM does not depend heavily on the relationship between the prime factors. Instead, its effectiveness is largely influenced by the size of the smallest prime factor.

When a factor is recovered, K1Wi can derive the remaining RSA parameters and continue RSA key recovery and ciphertext analysis. This makes ECM a valuable option when simpler factorization techniques are unsuccessful.

Investigation Notes

RSA-ECM is most effective against:

- RSA challenge keys containing a relatively small prime factor
- Educational cryptography exercises
- Capture-The-Flag (CTF) challenges
- Weakly generated RSA moduli

ECM is commonly used as an intermediate step between Pollard Rho and more computationally intensive factorization methods.

Practical Applications

RSA-ECM can be used to:

- Recover factors from vulnerable RSA challenge files
- Demonstrate modern factorization techniques
- Compare factorization strategies against the same modulus
- Support cryptography coursework and laboratory exercises
- Analyze intentionally weak RSA implementations

9.6 RSA-WIENER

Purpose

RSA-WIENER implements Wiener's continued fraction attack against RSA keys that use unusually small private exponents.

Unlike RSA-FACTOR, RSA-RHO, and RSA-ECM, Wiener's attack does not attempt to factor the RSA modulus directly. Instead, it exploits a mathematical weakness that occurs when the private exponent is significantly smaller than expected.

When vulnerable RSA parameters are used, Wiener's attack can recover the private exponent without factoring the modulus.

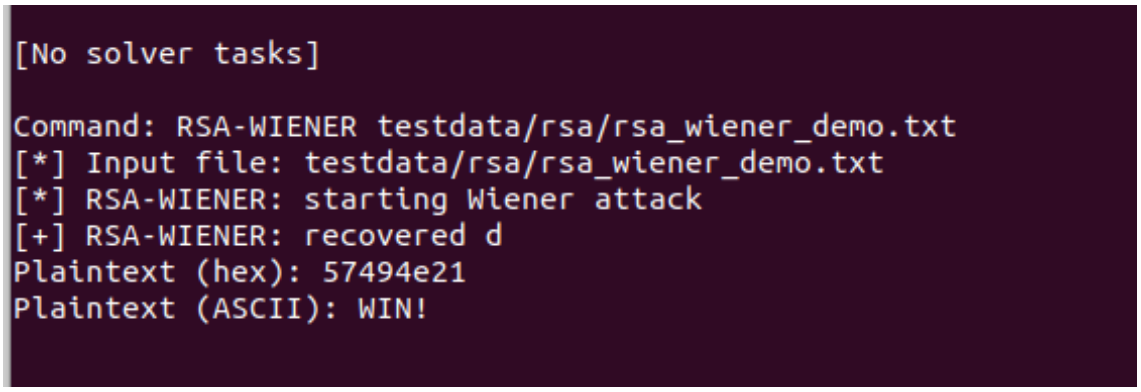
Usage

```
./bin/k1wi RSA-WIENER <rsa_file>
```

Example

```
./bin/k1wi RSA-WIENER testdata/rsa/rsa_wiener_demo.txt
```

Figure 9.4 – RSA-WIENER Private Exponent Recovery



RSA-WIENER using Wiener's continued fraction attack to recover RSA private exponents from weakly configured keys without factorizing the modulus.

Analyst Interpretation

Wiener's attack evaluates the mathematical relationship between the public exponent and the RSA modulus using continued fractions. Under certain conditions, an unusually small private exponent can be recovered directly from public key information.

Unlike factorization-based attacks, RSA-WIENER does not require recovery of the RSA prime factors. Instead, it exploits weaknesses in RSA parameter selection and attempts to derive the private exponent directly.

This approach makes RSA-WIENER particularly useful when factorization attacks are unsuccessful but the RSA key may have been generated using weak or intentionally vulnerable parameters. By targeting RSA configuration weaknesses rather than the modulus itself, Wiener's attack provides analysts with an alternative path to key recovery.

Investigation Notes

RSA-WIENER is most useful when:

- Factorization attempts fail
- The RSA modulus appears large but weakly configured
- A challenge specifically targets small private exponents
- Public key parameters suggest an unusual key structure

Because the attack does not require factorization, it can occasionally succeed against challenge files that resist RSA-FACTOR, RSA-RHO, and RSA-ECM.

Practical Applications

RSA-WIENER can be used to:

- Recover weak RSA private exponents
- Demonstrate continued fraction cryptanalysis
- Analyze intentionally vulnerable RSA keys
- Solve Capture-The-Flag (CTF) cryptography challenges
- Support educational cryptography exercises

9.7 RSA-SMALL-E

Purpose

RSA-SMALL-E implements a low public exponent attack against improperly configured RSA systems.

When RSA uses a very small public exponent, such as e = 3, and the plaintext is sufficiently small, encryption may be vulnerable to direct mathematical recovery without requiring factorization or private key recovery.

RSA-SMALL-E demonstrates this weakness and provides an educational example of how improper RSA parameter selection can compromise confidentiality.

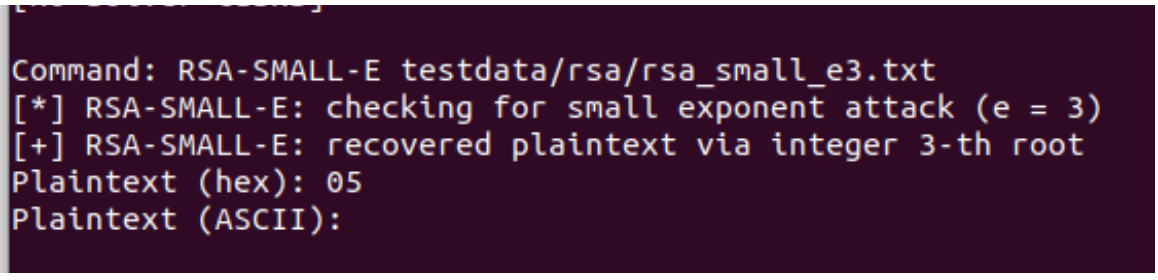
Usage

```
./bin/k1wi RSA-SMALL-E <rsa_file>
```

Example

```
./bin/k1wi RSA-SMALL-E testdata/rsa/rsa_small_e3.txt
```

Figure 9.5 – RSA-SMALL-E Plaintext Recovery



RSA-SMALL-E recovering plaintext from a vulnerable RSA challenge using a low public exponent attack.

Analyst Interpretation

RSA-SMALL-E evaluates whether the ciphertext can be recovered directly through integer root calculations rather than factorization or private key recovery.

When the public exponent is extremely small and the plaintext is insufficiently protected, the ciphertext may remain vulnerable to direct mathematical recovery. In these situations, the original message can sometimes be recovered without attacking the RSA modulus itself.

Unlike RSA-FACTOR, RSA-RHO, and RSA-ECM, this attack targets weaknesses in RSA parameter selection rather than weaknesses in the factorization problem.

Investigation Notes

RSA-SMALL-E is most effective when:

- The public exponent is very small
- Plaintext values are short or predictable
- No padding scheme is used
- Educational or challenge RSA parameters are present
- RSA is intentionally weakened for demonstration purposes

Modern RSA implementations avoid this weakness through the use of secure padding schemes such as OAEP.

Practical Applications

RSA-SMALL-E can be used to:

- Demonstrate low public exponent weaknesses
- Recover plaintext from vulnerable RSA challenges
- Analyze improperly configured RSA systems
- Support cryptography education and coursework
- Solve Capture-The-Flag (CTF) challenges involving weak RSA parameters

9.8 RSA-KNOWNPQ

Purpose

RSA-KNOWNPQ decrypts RSA ciphertext when the prime factors of the modulus are already known.

This module is useful after successful factorization, forensic recovery of RSA parameters, educational cryptography exercises, and Capture-The-Flag (CTF) challenges where the prime values have been provided or recovered through another attack.

Unlike RSA-FACTOR, RSA-RHO, and RSA-ECM, this module does not attempt to break RSA. Instead, it uses known prime factors to reconstruct the private key and decrypt ciphertext.

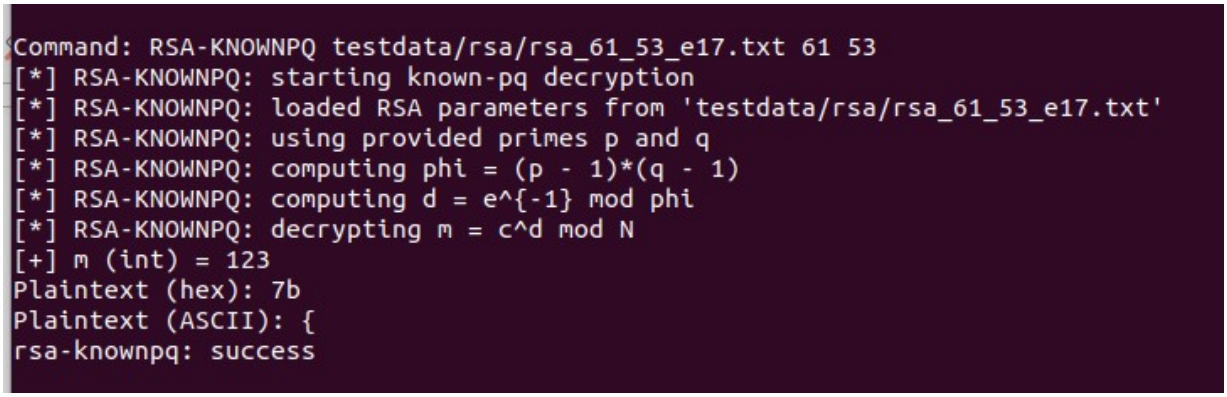
Usage

```
./bin/k1wi RSA-KNOWNPQ <rsa_file>
```

Example

```
./bin/k1wi RSA-KNOWNPQ testdata/rsa/rsa_61_53_e17.txt 61 53
```

Figure 9.6 – RSA Private Key Reconstruction



RSA-KNOWNPQ reconstructing an RSA private key and recovering plaintext using known prime factors.

Analyst Interpretation

RSA-KNOWNPQ assumes that the prime factors of the RSA modulus have already been recovered. Using the supplied values of p and q, the module reconstructs the RSA private key and decrypts the ciphertext.

This process demonstrates a fundamental property of RSA security: once the prime factors become known, the private exponent can be derived and the ciphertext can be decrypted. The command is therefore useful for validating factorization results and demonstrating RSA key reconstruction workflows.

Investigation Notes

RSA-KNOWNPQ is useful when:

- Prime factors have already been recovered
- An RSA challenge provides p and q directly
- Verifying the results of a factorization attack
- Recovering plaintext from educational RSA exercises
- Validating cryptography coursework solutions

Because the factors are already known, decryption is typically immediate.

Practical Applications

RSA-KNOWNPQ can be used to:

- Recover plaintext from RSA challenge files
- Verify factorization results
- Demonstrate RSA key reconstruction

- Support cryptography training exercises
- Analyze recovered RSA artifacts

9.9 RSA-CHECKPQ

Purpose

RSA-CHECKPQ validates candidate RSA prime factors.

This utility verifies whether supplied values are prime numbers and can be used to confirm factorization results before proceeding with RSA key reconstruction.

RSA-CHECKPQ does not perform factorization. Instead, it validates user-supplied values that may have been recovered through RSA-FACTOR, RSA-RHO, RSA-ECM, or other analysis methods.

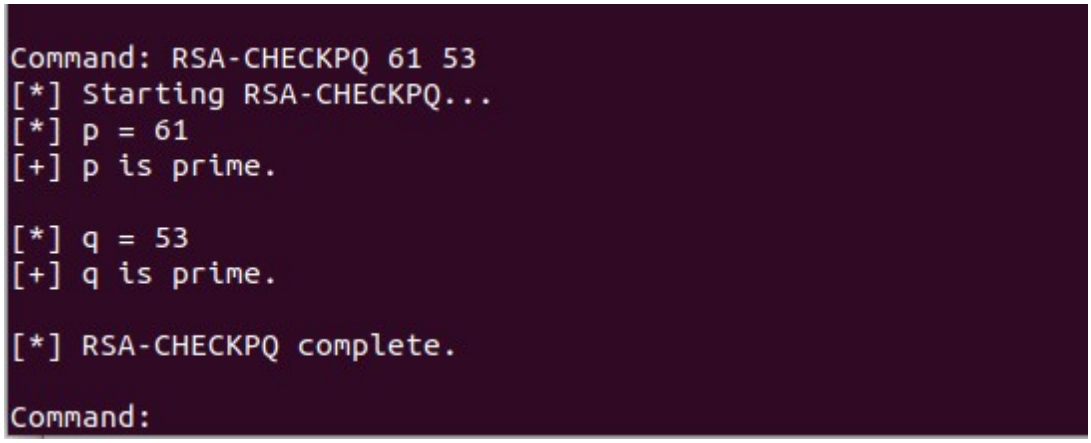
Usage

```
./bin/k1wi RSA-CHECKPQ
```

Example

```
./bin/k1wi RSA-CHECKPQ 61 53
```

Figure 9.7 – RSA Prime Validation



RSA-CHECKPQ validating candidate RSA prime factors prior to private key reconstruction.

Analyst Interpretation

RSA-CHECKPQ verifies whether the supplied values are valid prime numbers. Because RSA security depends on the use of prime factors, validation is an important step before computing Euler's Totient Function or reconstructing the private key.

This utility is commonly used after a successful factorization attempt to confirm that recovered values are suitable for RSA key recovery operations.

Investigation Notes

RSA-CHECKPQ is useful when:

- Verifying factorization results
- Validating RSA challenge solutions
- Confirming candidate prime values
- Supporting cryptography coursework
- Preparing for RSA private key reconstruction

Successful validation provides confidence that subsequent RSA recovery operations will produce correct results.

Practical Applications

RSA-CHECKPQ can be used to:

- Validate recovered RSA prime factors
- Verify challenge solutions
- Support RSA key reconstruction workflows
- Demonstrate prime validation concepts
- Assist with cryptography training exercises

9.10 RSA-DFROMPQ

Purpose

RSA-DFROMPQ computes the RSA private exponent using known values for the prime factors and public exponent.

This utility is useful when reconstructing RSA private keys, validating challenge solutions, verifying factorization results, and supporting cryptography coursework.

RSA-DFROMPQ does not perform factorization. Instead, it assumes the RSA prime factors have already been recovered and uses them to calculate Euler's Totient Function and the private exponent.

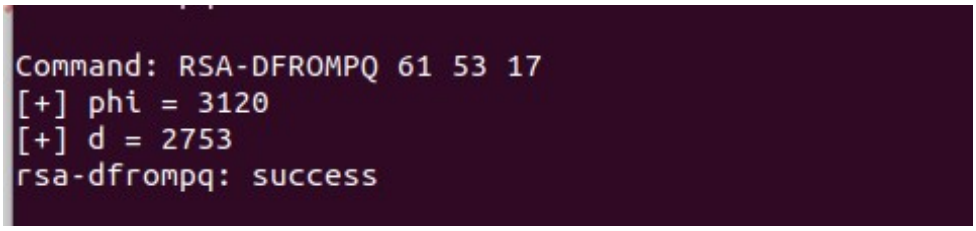
Usage

```
./bin/k1wi RSA-DFROMPQ
```

Example

```
./bin/k1wi RSA-DFROMPQ 61 53 17
```

Figure 9.8 – RSA Private Exponent Recovery



RSA-DFROMPQ computing Euler's Totient Function and recovering the RSA private exponent from known prime factors.

Analyst Interpretation

RSA-DFROMPQ uses the supplied prime factors and public exponent to compute Euler's Totient Function and derive the corresponding private exponent.

Once the prime factors of the RSA modulus are known, recovery of the private exponent becomes a straightforward mathematical operation. This utility allows analysts to verify factorization results, reconstruct RSA key material, and validate cryptographic calculations.

In this example, the recovered private exponent is:

d = 2753

which can then be used for RSA decryption and key reconstruction workflows.

Investigation Notes

RSA-DFROMPQ is useful when:

- Prime factors have already been recovered
- Validating RSA challenge solutions
- Demonstrating RSA key generation mathematics
- Reconstructing RSA private keys
- Supporting cryptography coursework and training exercises

Because no factorization is required, calculations are typically completed instantly.

Practical Applications

RSA-DFROMPQ can be used to:

- Recover RSA private exponents
- Verify factorization results

- Validate cryptographic calculations
- Demonstrate RSA key generation concepts
- Support RSA challenge analysis

9.11 RSA-MINI

Purpose

RSA-MINI is a lightweight educational RSA helper designed for small challenge files and introductory cryptography exercises.

The module demonstrates RSA attack concepts in a simplified environment and is intended primarily for learning, experimentation, and Capture-The-Flag (CTF) training scenarios.

Unlike the more advanced RSA modules, RSA-MINI focuses on demonstrating RSA weaknesses using intentionally vulnerable parameters.

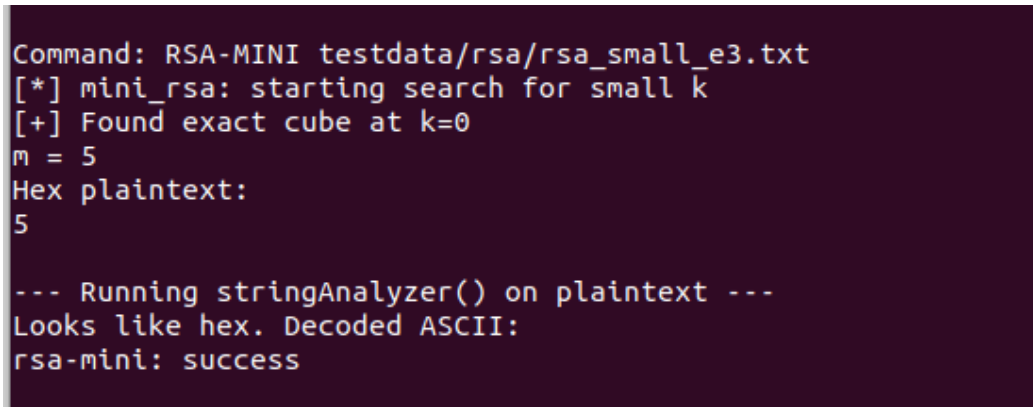
Usage

```
./bin/k1wi RSA-MINI <rsa_file>
```

Example

```
./bin/k1wi RSA-MINI testdata/rsa/rsa_61_53_e17.txt
```

Figure 9.9 – RSA-MINI Educational Workflow



RSA-MINI demonstrating RSA recovery concepts using a simplified educational challenge file.

Analyst Interpretation

RSA-MINI provides a simplified RSA analysis workflow designed for educational challenge files and introductory cryptography exercises.

The module is intended to help users understand RSA key relationships, weak RSA configurations, small exponent attacks, and basic RSA recovery concepts without requiring a detailed understanding of advanced cryptanalysis techniques.

Because RSA-MINI operates in a controlled educational environment, it is particularly useful for demonstrations, classroom instruction, and cryptography training exercises.

Investigation Notes

RSA-MINI is useful when:

- Learning RSA fundamentals
- Demonstrating small exponent weaknesses
- Practicing Capture-The-Flag (CTF) cryptography challenges
- Exploring RSA attack concepts in a controlled environment
- Supporting classroom instruction and training exercises

Current versions of RSA-MINI are intentionally limited in scope and should be viewed as an educational companion to the more advanced RSA analysis tools.

Practical Applications

RSA-MINI can be used to:

- Demonstrate RSA attack concepts
- Support cryptography coursework
- Validate educational challenge files
- Introduce RSA key recovery techniques
- Provide simplified examples for new users

9.12 Practical RSA Workflow

The K1Wi RSA modules are designed to operate together as a complete RSA analysis toolkit. While each module can be used independently, many cryptographic investigations follow a common recovery workflow.

The following example demonstrates a typical RSA challenge-solving process.

Step 1 – Factor the RSA Modulus

These modules attempt to recover the prime factors p and q used to generate the RSA modulus.

Begin by attempting to recover the RSA prime factors:

```
./bin/k1wi RSA-FACTOR testdata/rsa/rsa_61_53_e17.txt
```

If RSA-FACTOR is unsuccessful, additional factorization methods may be attempted:

```
./bin/k1wi RSA-RHO testdata/rsa/rsa_61_53_e17.txt
./bin/k1wi RSA-ECM testdata/rsa/rsa_61_53_e17.txt
```

These modules attempt to recover the prime factors p and q, which form the RSA modulus.

Step 2 – Validate Recovered Factors

Once candidate factors have been identified, verify that they are valid primes:

```
./bin/k1wi RSA-CHECKPQ 61 53
```

Successful validation confirms that the recovered values are suitable for RSA key reconstruction.

Step 3 – Recover the Private Exponent

After validating the prime factors, compute the private exponent:

```
./bin/k1wi RSA-DFROMPQ 61 53 17
```

This step reconstructs the RSA private key parameters.

Step 4 – Recover Plaintext

Once the RSA key has been reconstructed, decrypt the ciphertext:

```
./bin/k1wi RSA-KNOWNPQ testdata/rsa/rsa_61_53_e17.txt 61 53
```

K1Wi computes the private key and recovers the original plaintext.

Alternative Workflows

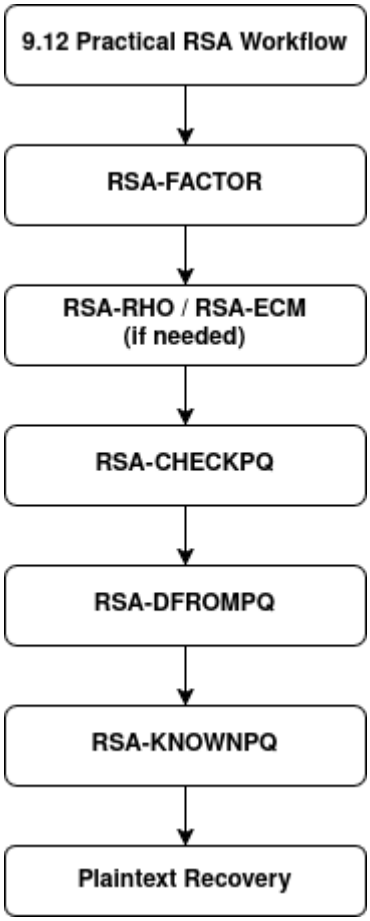
Not all RSA challenges require factorization.

Some challenge files may be vulnerable to RSA-WIENER when weak private exponents are present.

Others may be vulnerable to RSA-SMALL-E when insecure public exponent configurations are used.

In educational environments and Capture-The-Flag (CTF) competitions, analysts should evaluate multiple attack paths before selecting a recovery strategy.

Figure 9.10 – Complete RSA Analysis Workflow



Complete RSA recovery workflow using multiple K1Wi RSA analysis modules.

9.13 Summary

The K1Wi RSA Analysis Tool Suite provides a collection of educational and investigative modules for exploring RSA cryptography, recovering keys, validating parameters, and solving cryptographic challenges.

Throughout this chapter, the reader was introduced to RSA-FACTOR, RSA-RHO, RSA-ECM, RSA-WIENER, RSA-SMALL-E, RSA-KNOWNPQ, RSA-CHECKPQ, RSA-DFROMPQ, and RSA-MINI. Together, these modules demonstrate several important RSA concepts, including integer factorization, prime validation, private key reconstruction, weak parameter analysis, public-key cryptography fundamentals, and ciphertext recovery workflows.

The RSA modules are intended for educational analysis, cryptography research, coursework, and Capture-The-Flag (CTF) exercises. By combining multiple recovery techniques and supporting utilities, the K1Wi RSA suite provides practical examples of how weaknesses in RSA key generation, parameter selection, and implementation can affect cryptographic security.

Modern RSA implementations use large key sizes, carefully selected parameters, and secure padding mechanisms that protect against the attacks demonstrated in this chapter. As a result, the techniques presented here should be viewed as educational tools for understanding RSA security rather than methods for compromising properly implemented cryptographic systems.

With the completion of this chapter, the reader should now have a working understanding of the K1Wi RSA toolset and how the individual modules interact within a complete cryptographic analysis workflow. The RSA Analysis Tool Suite serves as both a practical learning platform and a foundation for exploring more advanced topics in cryptography and cryptanalysis.

The RSA modules demonstrate that cryptographic security depends not only on strong algorithms, but also on proper key generation, parameter selection, and implementation practices.

This concludes Chapter 9 – RSA Analysis Tools.

Appendix A – Glossary

The glossary provides authoritative definitions for major terms used throughout the K1Wi User Manual. It serves as a quick-reference resource for students, researchers, analysts, developers, and investigators who require precise definitions of forensic, cryptographic, reverse engineering, and binary analysis terminology.

The definitions contained in this appendix are intended to provide concise technical explanations rather than exhaustive academic treatments. Where applicable, terms are described in the context of their use within the K1Wi Framework.

A–C

ASCII

A 7-bit character encoding standard used for representing text. In K1Wi, ASCII detection is part of the STRING Intelligence Engine.

ASLR (Address Space Layout Randomization)

A security mechanism that randomizes memory layout to prevent predictable exploitation. Relevant to **PIECALC**.

Base64

A binary-to-text encoding scheme commonly used to represent binary data as printable characters. K1Wi detects Base64 candidates during STRING analysis.

Block-Level Entropy

Entropy computed over fixed-size blocks to reveal localized randomness anomalies.

Carving

The process of extracting embedded files based on signatures and structural heuristics. Implemented in the **File Carver** subsystem.

Ciphertext

Data that has been encrypted and is unreadable without the appropriate decryption process or cryptographic key.

Chi-Square Test

A statistical test used in JPEG steganalysis to detect deviations from expected pixel distributions.

CTF (Capture-The-Flag)

A cybersecurity competition that presents participants with technical challenges involving cryptography, reverse engineering, digital forensics, exploitation, and software analysis.

D–F

DCT (Discrete Cosine Transform)

The mathematical transform used in JPEG compression. DCT coefficients are analyzed by K1Wi during JPEG forensic and steganalysis workflows.

Designation Block

A structured metadata container used within K1Wi. Designation blocks can be inspected and validated using the DESIG family of commands.

Difference-of-Squares

A factorization technique used in RSA analysis when primes are close.

Dynamic Library

A shared library loaded by an executable at runtime. Dynamic libraries provide reusable code and reduce executable size.

ELF (Executable and Linkable Format)

The standard executable file format used by Linux and many Unix-like operating systems. ELF files contain executable code, metadata, symbol information, libraries, and runtime loading information. ELF files are analyzed using the ELFINFO module.

ELF Header

The primary structure located at the beginning of an ELF file. It contains metadata describing file type, architecture, entry point address, and offsets to additional ELF structures.

Elliptic Curve Method (ECM)

A factorization algorithm that uses arithmetic performed on elliptic curves to discover non-trivial factors of composite integers.

Entropy

A measure of randomness. Used extensively in forensic analysis.

Extraction Engine

The recursive subsystem responsible for multi-layer unwrapping of files.

Fermat Factorization

A method for factoring numbers where primes are close together.

Frequency Analysis

A classical cryptanalytic technique used in substitution cipher solving.

G–L

GMP (GNU Multiple Precision Library)

A high-precision arithmetic library used by K1Wi for RSA analysis, factorization, and cryptographic calculations.

Heatmap (Entropy)

A visual representation of block-level entropy values.

Hillclimbing

An optimization technique used in cryptanalysis to refine keys.

Huffman Table

A component of JPEG compression used to efficiently encode image data. Huffman tables can be analyzed during JPEG forensic investigations.

IC (Index of Coincidence)

A statistical measure used to estimate Vigenère key length.

JPEG Stego Engine

The subsystem responsible for JPEG steganalysis.

K1Wi

An integrated cybersecurity analysis framework that combines digital forensics, reverse engineering, cryptanalysis, binary analysis, and Capture-The-Flag (CTF) tooling into a unified command-line environment.

Key Length Estimation

The process of determining likely Vigenère key lengths using IC and Kasiski.

Least Significant Bit (LSB)

The lowest-order bit in a binary value. LSB modification is commonly used in steganography and steganalysis investigations.

M–P

Magic Bytes

Signature bytes used to identify file formats.

Modulus (N)

The product of two prime numbers used in RSA cryptography. The modulus forms part of both the public and private keys.

PIE (Position-Independent Executable)

A binary that can be loaded at any memory address. Analyzed by **PIECALC**.

Plaintext

The decoded or decrypted form of a ciphertext.

Pollard Rho

A probabilistic integer factorization algorithm commonly used to recover factors from weak RSA challenge files.

Private Exponent (d)

The RSA private key component used during decryption. The private exponent is derived from the RSA prime factors and public exponent.

Probability Model (Quadgrams)

The statistical model used to score plaintext candidates.

Program Header

A structure within an ELF file that describes how executable segments should be loaded into memory during program execution.

Q–S

Quadgram

A sequence of four letters used in statistical scoring.

Recursion Guard

A safety mechanism that prevents infinite extraction loops.

Regular/Singular (RS) Analysis

A steganalysis technique that detects LSB embedding.

Return Address

A memory address used in binary execution and software exploitation. Return addresses are commonly examined during reverse engineering and PIE analysis workflows.

RSA

A public-key cryptosystem based on integer factorization.

Section Header

A structure that describes individual sections contained within an ELF file, such as code, data, symbols, and debugging information.

Shannon Entropy

The mathematical measure of randomness used throughout K1Wi for forensic analysis, entropy calculations, and steganalysis workflows.

String Intelligence

The subsystem responsible for extracting ASCII, UTF-8, base64, URLs, and emails.

Symbol Table

A collection of named functions, variables, and addresses contained within an executable or object file. Symbol tables are commonly used during reverse engineering and debugging.

T–Z

UTF-8

A variable-length character encoding standard. Detected by the String Intelligence Engine.

Vigenère Cipher

A polyalphabetic substitution cipher solved by **VIG**.

Wiener's Attack

A cryptanalytic attack that exploits RSA implementations using unusually small private exponents.

Zip Archive

A compressed archive format commonly encountered in multi-layer extraction workflows.

Appendix B – Command Quick Reference

This appendix provides a quick reference for commonly used K1Wi commands. It is intended as a compact lookup guide for users who already understand the purpose of each module and need a fast reminder of command syntax.

Core Commands

Command

```
./bin/k1wi –version
```

Purpose

Displays the current K1Wi version, build date, and build information.

Command

```
./bin/k1wi HELP
```

Purpose

Displays the built-in help system and available command categories.

Command

```
./bin/k1wi MENU
```

Purpose

Displays the interactive K1Wi command menu.

String Analysis Commands

Command

```
./bin/k1wi STRING "text"
```

Purpose

Analyzes a directly supplied string for printable text, encoding indicators, entropy, and suspicious artifacts.

Command

```
./bin/k1wi STRING –file
```

Purpose

Analyzes strings extracted from a file.

Command

```
./bin/k1wi STRING --decode
```

Purpose

Attempts to decode supported encoded content.

File and Forensic Analysis Commands

Command

```
./bin/k1wi ENTROPY <file>
```

Purpose

Calculates entropy for the supplied file.

Command

```
./bin/k1wi LYZER
```

Purpose

Performs image and file forensic analysis using the default LYZER workflow.

Command

```
./bin/k1wi LYZER <file> ALL
```

Purpose

Runs the full LYZER forensic analysis workflow, including entropy analysis, embedded signature detection, file carving, string intelligence, and steganography heuristics.

Command

```
./bin/k1wi EXTRACT
```

Purpose

Extracts supported content from an archive or container file.

Command

```
./bin/k1wi EXTRACT –recursive
```

Purpose

Performs recursive extraction against nested archives and discovered artifacts.

Binary and Reverse Engineering Commands

Command

```
./bin/k1wi ELFINFO -IN
```

Purpose

Displays ELF header information, program headers, dynamic libraries, and symbol tables for a Linux ELF binary.

Command

```
./bin/k1wi PIECALC --bin –list
```

Purpose

Lists available symbols and offsets from a PIE-enabled binary.

Command

```
./bin/k1wi PIECALC --bin –nearest
```

Purpose

Identifies the nearest known symbol to a supplied address.

Command

```
./bin/k1wi PIECALC --bin --json –list
```

Purpose

Generates symbol information in JSON format for scripting and automation.

RSA Analysis Commands

Command

```
./bin/k1wi RSA-FACTOR <rsa_file>
```

Purpose

Attempts to factor an RSA modulus and recover RSA private key parameters.

Command

```
./bin/k1wi RSA-RHO <rsa_file>
```

Purpose

Runs Pollard Rho factorization against a vulnerable RSA challenge file.

Command

```
./bin/k1wi RSA-ECM <rsa_file>
```

Purpose

Runs Elliptic Curve Method factorization against an RSA challenge file.

Command

```
./bin/k1wi RSA-WIENER <rsa_file>
```

Purpose

Attempts Wiener's continued fraction attack against RSA keys with unusually small private exponents.

Command

```
./bin/k1wi RSA-SMALL-E <rsa_file>
```

Purpose

Attempts plaintext recovery against RSA configurations using weak low public exponents.

Command

```
./bin/k1wi RSA-KNOWNPQ <rsa_file>
```

Purpose

Uses known RSA prime factors to reconstruct the private key and decrypt ciphertext.

Command

```
./bin/k1wi RSA-CHECKPQ
```

Purpose

Validates candidate RSA prime factors.

Command

```
./bin/k1wi RSA-DFROMPQ
```

Purpose

Computes Euler's Totient Function and derives the RSA private exponent from known p, q, and e values.

Command

```
./bin/k1wi RSA-MINI <rsa_file>
```

Purpose

Runs a simplified educational RSA helper for small challenge files.

Hashing and File Identification Commands

Command

```
./bin/k1wi MAGIC
```

Purpose

Identifies file type information using magic byte detection.

Command

```
./bin/k1wi MD5
```

Purpose

Computes the MD5 hash of a file.

Command

```
./bin/k1wi MD5 -verify
```

Purpose

Verifies a file against a supplied MD5 hash.

Command

```
./bin/k1wi MD5 -compare
```

Purpose

Compares two files using MD5 hashes.

Command

```
./bin/k1wi SHA256
```

Purpose

Computes the SHA-256 hash of a file.

Command

```
./bin/k1wi SHA256 -verify
```

Purpose

Verifies a file against a supplied SHA-256 hash.

Command

```
./bin/k1wi SHA256 -compare
```

Purpose

Compares two files using SHA-256 hashes.

Testing and Build Commands

Command

```
make clean
```

Purpose

Removes previous build artifacts.

Command

```
make
```

Purpose

Compiles the K1Wi Framework.

Command

```
make debug
```

Purpose

Builds K1Wi with debugging symbols and development diagnostics.

Command

```
make release
```

Purpose

Builds K1Wi using release-oriented compiler settings.

Command

```
make sanitize
```

Purpose

Builds K1Wi with AddressSanitizer instrumentation.

Command

```
make tsan
```

Purpose

Builds K1Wi with ThreadSanitizer instrumentation.

Command

```
./tests/run_regression.sh
```

Purpose

Runs the K1Wi regression test suite.

Recommended First Commands

New users should begin with the following commands:

```
./bin/k1wi -version
./bin/k1wi HELP
./bin/k1wi STRING "hello world"
./bin/k1wi ENTROPY <file>
./bin/k1wi LYZER <file> ALL
./bin/k1wi ELFINFO -IN ./bin/k1wi
./bin/k1wi RSA-FACTOR testdata/rsa/rsa_61_53_e17.txt
```

Summary

This quick reference provides a compact command overview for common K1Wi workflows. For detailed explanations, examples, screenshots, and analyst interpretation, refer to the corresponding chapters in the main manual.